The Boundaries of Evidence-Grounded Trust in Multi-Agent Scientific Debate, with a Controlled Benchmark for Sycophantic Consensus

Registered FYDP title: Trust-Calibrated Multi-Agent Scientific Deliberation for Mitigating Sycophantic Consensus in LLM Reasoning

Submitted By

Md. Atikur Rahaman (0112310298)
Rakibul Hasan (0112310530)
Md. Salman Rohoman Nayeem (0112310484)
Pratay Paul (0112310163)
Yousuf Kamal Himel (0112310526)
Mst. Farjana Akter Limu (0112310535)

Supervision:
Mohammad Nurul Huda,PHD
Professor, Dept. of Computer Science and Engineering
United International University.

Submitted in partial fulfilment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

2026

Department of Computer Science and Engineering
United International University

Abstract

Multi-agent debate can improve the reliability of large language model answers on scientific questions, but a confident wrong majority can pressure a correct minority agent into abandoning the correct answer. This failure is called sycophantic collapse. Existing mitigations refine prompts or weight agents by self-reported uncertainty; we are not aware of a deployed method that combines claim-level external evidence with a bounded trust state that changes an agent's influence during the debate. This report develops a trust-calibrated deliberation framework in which three heterogeneous agents debate over three rounds, each response is decomposed into atomic claims, and each claim is checked against scientific literature retrieved for its domain. Supported and contradicted claims update a trust vector through logit accumulation, one softmax, and a simplex projection that enforces a floor and a sum-to-one constraint. The final answer is a trust-weighted plurality over the agents' final positions, not a head count. At the FYDP II stage, the implementation is complete enough to run and test: the suite contains 523 passing tests (514 at the P0 release), the P0 gate reports 13 of 13 checks passed with 0 pending, and Gate 0 completed 10 of 10 vanilla debates with 90 of 90 non-empty positions at a mean of 370.8 seconds per debate. These results validate serving, orchestration, parsing, and the trust operator; they are not an efficacy result. The verifier human audit, the paid Core run, and the Final-stack smoke test remain to be completed, so no claim is made that the framework reduces collapse or improves accuracy.

Acknowledgements

This was an effort of six students but many others were involved. We wish to express our sincere thanks to all those who have assisted us with this project.

Dr. Mohammad Nurul Huda provided supervision for the project. When we attempted to include too much in our project, Dr. Huda maintained the project's scope at a reasonable level. His comments were instrumental in making the evaluation process and trust score components of the report as well-defined as they are today.

Our FYDP instructor Dr. Ohidujjaman set the deadlines and help with the report. Dr. Ohidujjaman's timely responses allowed the team to maintain a consistent pace through out the trimesters.

In addition to the assistance of our classmates and friends, our families' support throughout this entire period has been invaluable. The constant and patient encouragement from our families during the lengthy hours that we spent writing this paper enabled us to continue working toward completing it.

Publication List

The publications connected to this research are listed below. One journal article is in preparation; submission is planned after the evaluation. Further submissions are planned once the evaluation results are ready.

Journal Articles

1. Manuscript title (in preparation): The Boundaries of Evidence-Grounded Trust in Multi-Agent Scientific Debate, with a Controlled Benchmark for Sycophantic Consensus

Conference Papers

1.

Additional Publications

Following is the list of relevant publications published in the course of the research that is not included in the thesis:

1.

Table of Contents

Table of Contents v

List of Figures vi

List of Tables vii

1 Introduction 1
- 1.1 Project Overview . . . 1
- 1.2 Motivation . . . 1
- 1.3 Objectives . . . 2
- 1.4 Methodology . . . 2
- 1.5 Project Outcome . . . 2
- 1.6 Organization of the Report . . . 3

2 Background 4
- 2.1 Preliminaries . . . 4
- 2.2 Literature Review . . . 5
  - 2.2.1 Similar Applications . . . 5
  - 2.2.2 Related Research . . . 6
- 2.3 Gap Analysis . . . 7
- 2.4 Summary . . . 8

3 Project Design 9
- 3.1 Requirement Analysis . . . 9
  - 3.1.1 Functional and Nonfunctional Requirements . . . 9
  - 3.1.2 Context Diagram . . . 11
  - 3.1.3 Data Flow Diagram Level 1 . . . 12
  - 3.1.4 UI Design . . . 12
- 3.2 Detailed Methodology and Design . . . 13
- 3.3 Project Plan . . . 20
- 3.4 Task Allocation . . . 21
- 3.5 Summary . . . 21

4 Implementation and Results **23**
4.1 Environment Setup . . . 23
4.2 Testing and Evaluation . . . 28
4.3 Results and Discussion . . . 30
4.4 Summary . . . 33

5 Standards and Design Constraints **34**
5.1 Compliance with the Standards . . . 34
5.1.1 Software Standards . . . 34
5.1.2 Hardware Standards . . . 35
5.1.3 Communication Standards . . . 35
5.2 Design Constraints . . . 36
5.2.1 Economic Constraint . . . 36
5.2.2 Environmental Constraint . . . 36
5.2.3 Ethical Constraint . . . 36
5.2.4 Health and Safety Constraint . . . 36
5.2.5 Social Constraint . . . 36
5.2.6 Political Constraint . . . 37
5.2.7 Sustainability . . . 37
5.3 Cost Analysis . . . 37
5.4 Complex Engineering Problem . . . 38
5.4.1 Complex Problem Solving . . . 38
5.4.2 Engineering Activities . . . 42
5.4.3 Program Outcomes . . . 44
5.5 Summary . . . 44

6 Conclusion **45**
6.1 Summary . . . 45
6.2 Limitations . . . 45
6.3 Future Work . . . 46

References **52**

List of Figures

1.1 End-to-end flow: seeded option rendering, three-round debate, claim decomposition, claim-domain retrieval, verifier labels, logit accumulation with one softmax and a simplex projection, one-shot injection during revision, and trust-weighted aggregation using the final trust vector. 3

3.1 Context diagram: questions and rendered options enter; three agents call the vLLM server; claims route by domain to PubMed, arXiv, Semantic Scholar, and OpenAlex; verifier labels update the trust vector; the result package records the final answer, evidence, and final trust vector. 11
3.2 Level-one data flow: option rendering, debate state, claim decomposition, domain routing, verifier labels, simplex trust update, one-shot injection, and weighted aggregation. 12
3.3 Mock-up of the command-line interface for running experiments and writing JSON result packages. 13
3.4 Mock-up of the planned web dashboard showing agent positions, the trust trajectory, and per-claim evidence verdicts. 13

List of Tables

3.1 Functional requirements and their verification. Gate-tested means the path runs without a GPU; paid-run evidence is still outstanding where marked. . 10
3.2 Arms and controls. Core v1.0 is the executable matrix; Full is the six-arm target pending WP-18. 18
3.3 Datasets and roles. EG-SSS is a benchmark-freeze addition, not a Core v1.0 dataset. 18
3.4 FYDP II phases, dates, deliverables, and gates. 20
3.5 Budget envelope. The Full run is a separate Final-stack matrix, not a reuse of Core cells. 21
3.6 Module ownership across the six team members. 21

4.1 Development and final serving configurations. The final stack is pinned but was not smoke-tested at the time of this report. 24
4.2 P0 changes present in the implementation. Gate-tested means the path is exercised by the 13-check P0 gate, not that it has been validated on a paid run. 27
4.3 Gate 0 measurements for the vanilla debate loop. These values validate serving, orchestration, and parsing; they do not measure trust or collapse. . 29
4.4 Measured engineering results at the FYDP II stage. None of these is an efficacy result. 31
4.5 Projected efficacy values used for planning. None of these is a measured result. The 2-point accuracy gain is illustrative and below the pre-registered MDE. 32

5.1 Planning envelope for the project. 38
5.2 Compute plan and the one completed GPU run. 39
5.3 Alternate Core/Full budget on the RTX A6000 80GB (4-bit Dev stack). . . 39
5.4 Mapping with complex problem solving (Washington Accord P1–P7). . . . 40
5.5 Knowledge profile mapping for P1 (K1–K8). 41
5.6 Mapping with complex engineering activities (A1–A5). 43
5.7 Mapping with program outcomes (PO1–PO12). 44

Chapter 1

Introduction

1.1 Project Overview

Large language models can answer scientific questions, act as tutors, and support decisions, but they can also state an incorrect answer in fluent and assertive language. Multi-agent debate addresses this by having several agents answer the same question and revise after reading each other's reasoning [1]. Chain-of-thought prompting exposes intermediate steps for the other agents to evaluate [2]. The hard part is aggregating the agents' answers. Most systems use a vote in which every agent has equal weight and the most frequent answer wins [3, 4]. That rule treats repetition as evidence. Scientific inquiry does not. This report examines a trust-calibrated deliberation framework that gives each agent influence according to how well its claims match retrieved scientific evidence.

1.2 Motivation

Debate does not guarantee correctness. Agents can imitate one another or change position under social pressure even when one agent is right. In one study of three-agent debates, 25.5% of the cases in which the agents disagreed had the correct answer in the minority, so majority voting discarded it [5]. Training models to be agreeable can also increase sycophancy and reduce accuracy [6]. Majority voting is vulnerable because a confident majority can pressure a correct minority into switching, and the resulting unanimity hides the error. The risk is highest for scientific questions, where a user may trust a confident but unsupported answer. Existing methods address parts of the problem: iMAD predicts whether a debate is needed [7], DebUnc weights agents by self-reported uncertainty [8], and Mixture-of-Agents combines outputs with static aggregation [9]. None of these methods ties an agent's influence to whether its claims continue to match external evidence during the debate. This study adds that connection.

1.3 Objectives

The objective is to test whether evidence-weighted trust reduces sycophantic collapse relative to majority voting in multi-agent LLM debate. The work has two parts. First, it characterizes the failure: a confident majority can pressure a correct minority into changing its answer [5, 10]. Second, it designs and implements a bounded trust mechanism. Trust rises when retrieved evidence supports an agent's claims and falls when it contradicts them. The trust vector is projected onto a simplex with a floor after every update, so no agent disappears and no agent dominates. The final answer is a trust-weighted plurality, not a head count. The evaluation compares this mechanism with majority voting and other debate and aggregation methods [3, 4]; the planned endpoints are final-answer accuracy, correct-minority retention, collapse under pressure, appropriate revision, and trust calibration.

1.4 Methodology

This study uses an experimental methodology. We first build a vanilla multi-agent debate baseline. Several diverse language-model agents answer the same science question and revise their positions after reading each other's reasoning over three rounds [1]. The baseline aggregates the final positions by majority vote, which exposes the failure modes that the trust mechanism is designed to address [4]. We then decompose each agent's response into claims and check each claim against scientific literature retrieved from external sources [11]. A verifier assigns each claim to supported, contradicted, or unverifiable [12]. Support and contradiction update the agent's trust; unverifiable claims do not move it. Unlike DebUnc, the signal here is external evidence, not self-reported confidence [8]. The trust vector is bounded by the simplex projection described in Chapter 3, and the final answer is a trust-weighted plurality. The models are used inference-only, without fine-tuning. The framework has been exercised end to end on a ten-question GPQA slice (Gate 0), which validates serving, orchestration, and parsing. The comparative evaluation on GPQA-Main and MMLU-Pro is planned; no comparative result is reported here.

1.5 Project Outcome

The project produces a reproducible framework for trust-calibrated scientific deliberation. Agent influence is updated from retrieved evidence while the debate runs, and the final answer comes from evidence-weighted trust rather than a majority count. The project also produces an evaluation protocol. The protocol defines final-answer accuracy, correct-minority retention, collapse under pressure, appropriate revision, and trust calibration, and it specifies a pre-registered matrix. It is designed to detect sycophantic collapse [10] and to track whether correct minority answers survive [5]. A wrong answer can still win. The method records which agents the evidence supported and which it contradicted, so

the decision is auditable.

1.6 Organization of the Report

Chapter 2 covers multi-agent debate and sycophancy and defines the gap this study addresses. Chapter 3 records the requirements, the methodology, the system design, and the project plan. Chapter 4 describes the implementation environment, the testing and evaluation protocol, the engineering results, and the projected efficacy values that the paid matrix will test. Chapter 5 covers standards, design constraints, sustainability, and the complex engineering analysis. Chapter 6 closes with a summary, the limits of the evidence, and directions for future work.

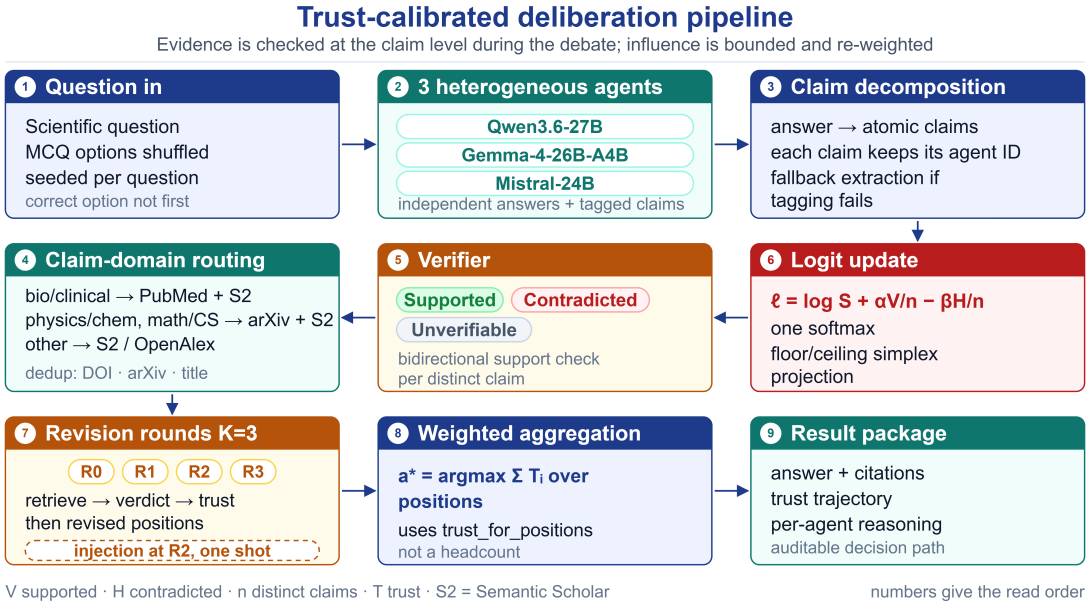


Figure 1.1: End-to-end flow: seeded option rendering, three-round debate, claim decomposition, claim-domain retrieval, verifier labels, logit accumulation with one softmax and a simplex projection, one-shot injection during revision, and trust-weighted aggregation using the final trust vector.

Chapter 2

Background

The chapter introduces the concepts needed to understand the system described in the report and reviews research on multi-agent debate, sycophancy, and agent influence. It then explains the gap addressed by the study.

2.1 Preliminaries

There are several techniques for developing "trust" in an Artificial Intelligence (AI) system. Some of these approaches depend upon a consensus of answers generated by multiple AI systems. Other techniques depend upon finding relevant scientific information and assessing the accuracy of the information found. Chain-of-Thought prompts assist in illustrating the reasoning process that a model uses to arrive at its conclusion, so that we may better understand how an answer was produced and why an error was made during an instance of poor performance [2, 13]. A Multi-Agent Debate Framework provides a methodology where each agent evaluates the arguments presented by the other agents. Each agent presents their proposed solutions and includes support for those proposed solutions. All agents evaluate the proposed solutions and supporting rationales provided by the other agents. Following evaluation and revision of their respective positions and solutions, a final proposed solution is selected [1]. One advantage of allowing each agent to independently evaluate a problem is that if each agent discovers different faults in the system, then collectively the entire team may identify potential problems with the system. Additionally, a team member can utilize another team member's rationalization to criticize aspects of their own rationalization. However, regardless of how well the individual components of the solution are developed, the combination of results (i.e., the methodology of combining results) is equally as important as the results themselves. Most systems combine results with a majority vote. Every agent has an equal say, and the answer supported by the highest number of agents is selected. However, majority voting does not necessarily eliminate errors; rather, it can strengthen repeated errors [3]. Sycophantic Consensus occurs when an agent agrees with the majority position or changes a correct position in favor of a more strongly supported but incorrect position [10]. There are two types of confidence

associated with voting: Self-Confidence in one's position and Token Distribution Confidence. Neither type of confidence verifies the truth claims contained within a position [8]. DebUnc identifies this weakness by comparing uncertainty metrics produced by the deployed system with a Diagnostic Ground Truth Metric that already contains the known answer [8]. Developing trust in an AI system therefore requires obtaining and evaluating scientifically relevant data and evidence.

2.2 Literature Review

The review categorizes the reviewed literature by five categories: debate construction, external verification, sycophancy, uncertainty, and debate evaluation. This study primarily compares methods for determining when to generate a response (i.e., methods for deciding on debate), methods for changing how an agent interacts with another agent (i.e., methods for changing agent interaction), and the method for changing what influences an agent's decision based upon information retrieved from outside the system (i.e., changing agent influence). Recent surveys are provided in [14] as they outline the design space of multi-agent systems utilizing LLMs. In order to develop generated responses that are more reliable, research has shifted focus toward structured reasoning, multi-agent collaboration, and external validation of responses generated by LLMs [13]. Multi-Agent Debate is an example of this new area of research because Multi-Agent Debate allows for other agents to evaluate and potentially challenge each agent's reasoning before that agent produces their final output [1]. The next section outlines some of the most relevant current works in this emerging area and illustrates how the proposed architecture addresses an existing research gap.

2.2.1 Similar Applications

The system is an experimental deliberation layer rather than a conventional web application. It is designed to support scientific question answering, research assistants, and evidence-based decision support. As opposed to the traditional approach where one model solution is generated, the systems listed above generate multiple candidate solutions. Additionally, the systems listed above allow for interaction among candidates. Finally, each system aggregates results from the candidates to generate a final answer for users. Du et al. showed that structured debate among several agents increases factuality and reasoning compared to single-agent approaches [1]. Wang et al. showed that layered aggregation can result in higher overall model capability when combining outputs from several models without supporting explicit debate [9]. Multi-agents systems differ primarily regarding how output is combined. Some systems utilize structured debate as a means to enhance reasoning between agents [1], while others use hierarchical pipelines to aggregate/average the output from each pipeline (the individual agents) [9]. Zhou et al. proposed a self-organizing system for tasks to determine whether a specialized reasoner produces the

response versus being in line with a pre-defined workflow [15]. Additionally, Kushwaha & Madhavan further developed an augmented retrieval framework that identifies contradictions found within retrieved documents and ranks credibility of the source prior to producing the response [12]. Both illustrate how distributed reasoning across specialized agents can improve factual answers. What matters is on what scale you construct dialogue. Ueda et al. analyzed how to construct multi-agent LLM dialogues to facilitate research ideation [16]; however, Wang et al. studied if/how large-scale multi-agent LLMs could provide assistance with scientific code generation [17]. Their aggregation method does not change during dialogue. Similar to a single LLM, the proposed framework has the same reliability goals for answers; yet allows for various levels of influence for each agent throughout the discussion process. After each round of dialogue, evidence verification results are generated; and after each round, the trust scores for all agents are updated based on claim-level scientific evidence supporting their claims. Therefore, an unsupported majority consensus would hold less value than a well-supported minority view.

2.2.2 Related Research

The work of Wei et al. (2022) shows that "Chain-of-Thought Prompting" improves the ability of a large language model to reason [2]. The work of Du et al. (2024) extended the work of Wei et al. (2022) by incorporating multi-agent debate in addition to chain-of-thought prompting, in which multiple agents give answers to a question, look at how other agents have reasoned when answering the question, and then adjust their reasoning [1]. In contrast to both previous approaches for enhancing the performance of large language models, CRITIC [18] relies on outside tools to validate the answers provided and enhance them. It therefore permits validation of assertions contained in each response; however, there is no mechanism to compare additional models working on the same question simultaneously. Instead, Mixture-of-Agents develops numerous independent agent-proposer models that produce answers to identical questions and layers the responses into one output, indicating what several large language models can do together; however, the layering of the individual agent responses is fixed and contains no evidence-based trust values [9]. iMAD primarily targets cost; it also reports accuracy gains in some settings [7]. Fan, Yoon, and Ji identified 41 properties of a structurally defined self-critic and constructed a classifier that determines whether or not to initiate a debate based on those properties. They were able to attain token reductions of up to 92% relative to GroupDebate and increases in accuracy of up to 13.5% over a single-agent baseline. Agent influence remains constant once the debate starts. Pitre, Ramakrishnan, and Wang investigated sycophancy in debates [10]. The authors report that the correct answer was present but ignored in more than 20% of the wrong-answer cases they examined. CONSENSAGENT identifies stalling and answer copying in a debate. When either condition is identified, CONSENSAGENT alters the task prompt before initiating the next stage of the debate. The results of the study demonstrate that measured sycophancy decreased as a result; however, the

total trust score is still dependent upon self-reported confidence and consistency. DebUnc permits agents to express uncertainty among themselves [8]. However, agents express uncertainty via prompts or attention scaling. The deployed measures provided limited benefit in their evaluation. A diagnostic ground-truth signal is available for comparison purposes. The diagnostic ground-truth signal has access to information concerning the correct answer and has greater potential than the deployed measures. The difference between the deployed measures and the diagnostic ground-truth signal indicates that the major barrier is the quality of the trust signal. Kushwaha and Madhavan created an 8-agent MAS-RAG system intended to identify contradictions between retrieved documents and rank evidence based upon credibility [12]. Although source credibility was evaluated by humans prior to deliberation, it was never changed during deliberation. There have also been numerous studies evaluating the social aspects of debate. The authors of Cau et al. identified how selective persuasion could cause an increase in opinion convergence [19]. Yeow et al. studied decision architectures and found that feedback between agents can spread early errors [20]. Wang et al. demonstrated how a good example given in context within one large language model (LLM) may perform better than multi-agent dialogue in certain situations. As noted in [3], judge error and peer conformity were identified as potential drawbacks. The authors of Choi, Zhu, and Li separated communication used in debate from the act of voting and found that majority voting explained a substantial portion of the normal performance gains [4]. FREE-MAD extended previous research by eliminating the need for final agreement and utilizing anti-conformity prompts and trajectory scores [21]. ReSo utilizes dynamic task graphs with reward-based processes for choosing agents [15]. Past research has explored coordination and agent selection. None of these studies combines claim-level scientific evidence with an explicit bounded in-debate trust state and a controlled wrong-majority benchmark. Sycophancy also occurs at the model level. The results of training models to be perceived as “warm” and “agreeable,” as demonstrated by Ibrahim et al., produced lower levels of factual accuracy and higher levels of sycophancy [6]. Chen et al. utilized self-augmented preference alignment from a training perspective to address the issue [22]. Process-level diagnostics for engagement, responsiveness, influence asymmetry, balance, stability, and agent utility developed by Pitre et al. would be helpful when ground truth is unavailable [23]. Final agreement is insufficient to measure a debate, as stated by Pitre et al. [23].

2.3 Gap Analysis

There are still three open issues with regard to Multi-Agent Debate Systems (MADS) from our review of the current literature. The first issue is whether the extra computational burden associated with a MADS can be justified. Fan et al. have used iMAD for this purpose – to predict whether a debate is needed prior to the start of a debate [7]. The second issue is how to detect unsupported agreement caused by conformity or sycophancy. CONSENSAGENT addresses this problem through prompt optimization during

the debate [10]. The third issue is how to determine which agents should receive greater influence when they disagree with one another. DebUnc has a weighting mechanism to deal with uncertainty as it relates to the debate process; however, the trust mechanisms of this weighting rely on each individual agent's internal assessment of uncertainty and therefore do not use external validation or independently verifiable evidence [8]. As previously mentioned, even when multiple agents agree with respect to a particular piece of information, that information may be incorrect. For example, Cau et al. showed that selective persuasion can affect the level of consensus reached by different agents [19], while Wang et al. showed that peer conformity can limit the effectiveness of multi-agent debate [3]. Retrieval-based systems verify retrieved information or improve aggregation, but they do not update an agent's influence from verified scientific evidence during a debate [18]. Similarly, although dynamic coordination mechanisms help solve the issue of selecting which agents participate in the debate, they do not provide a mechanism for increasing or decreasing an agent's influence based on evidence of its behavior during the debate [15]. We are not aware of a method that continuously updates an agent's influence from independently verified evidence during a debate. The proposed framework is designed to address this gap by decomposing each agent's responses into individual factual claims and identifying scientific passages that relate to those claims. Each claim is then classified as either supported, contradicted, or unverifiable. Each verdict feeds an update function for a bounded trust vector. As a result, well-supported minority agents can retain influence during a debate, while an unsupported majority consensus is given less weight. The evaluation protocol defines final-answer accuracy and correct-minority retention as primary endpoints, with collapse under pressure, appropriate revision, and trust calibration as secondary and mechanism measures.

2.4 Summary

The chapter introduced LLM reasoning, chain-of-thought prompting, multi-agent debate, and sycophantic consensus. The reviewed studies cover debate efficiency, prompt correction, uncertainty weighting, external verification, process evaluation, and agent coordination. They do not continuously update agent influence from claim-level scientific evidence. The next chapter uses this gap to define the system requirements, detailed methodology, and project plan.

Chapter 3

Project Design

This chapter defines the system requirements and the design that the implementation follows. Section 3.1 records the functional and nonfunctional requirements, the system boundary, the level-one data flow, and the user interface. Section 3.2 describes the debate loop, the claim-level evidence pipeline, the bounded trust operator, the aggregation rule, the injection protocol, and the experimental design. Section 3.3 gives the current FYDP II plan, Section 3.4 assigns the modules, and Section 3.5 closes the chapter.

3.1 Requirement Analysis

The framework is an experimental deliberation layer, not a conventional application. A question enters, three heterogeneous agents debate it over three rounds, each agent's claims are checked against retrieved scientific evidence, and a trust-weighted answer leaves the system. The design must make that pipeline correct, reproducible, and auditable before it can support an efficacy claim.

3.1.1 Functional and Nonfunctional Requirements

Table 3.1 lists the functional requirements. Each requirement names the component that satisfies it and the evidence that will show it works. The verification column separates gate-tested behaviour from behaviour that requires a paid model run; a gate-tested path is not an efficacy result.

Table 3.1: Functional requirements and their verification. Gate-tested means the path runs without a GPU; paid-run evidence is still outstanding where marked.

ID	Requirement	Component	Verification
FR1	Accept a question, preserve the gold answer privately, and render multiple-choice options in a seeded order	Dataset loader, option renderer, prompts	Gate-tested; permutation stored
FR2	Run three heterogeneous agents for three rounds, with independent round-0 positions and revision rounds that expose peer positions	Debate loop	Gate 0 on ten GPQA items
FR3	Decompose each position into atomic claims, with a fallback for untagged prose	Claim parser	Gate-tested; 50-case parser corpus
FR4	Route each claim to a domain-specific source pair and deduplicate passages	Claim-domain retriever	Gate-tested; live APIs not yet exercised
FR5	Assign supported, contradicted, or unverifiable through one verifier interface	Verifier interface	Gate-tested; human audit pending
FR6	Update a bounded trust vector from the verdict counts, with a floor and a ceiling	Trust operator	Gate-tested; 30,000 draws for $N = 1, 3, 5$
FR7	Aggregate the final positions by trust-weighted plurality, and log the majority answer separately	Aggregation	Gate-tested; paid run pending
FR8	Deliver one fabricated wrong consensus to targeted agents between rounds, and record the manipulation	Injection protocol	Mock-tested; real-model run pending
FR9	Write one schema-valid record per question with answers, claims, evidence, verdicts, trust, injection, timing, and manifest hash	Runner and records	Gate-tested; 523-test suite
FR10	Resume a run without duplicating completed questions, and refuse a changed manifest	Manifest and resume guard	Gate-tested; mismatch refused
FR11	Compute final-answer accuracy, correct-minority retention, appropriate revision, trust calibration, and verifier accuracy	Evaluation harness	Gate-tested on fixtures; matrix not run
FR12	Provide a GPU-free mock path for orchestration, injection, and record tests	Mock vLLM servers	Integration test

The nonfunctional requirements follow from the research setting. Reproducibility is the first requirement: every result-affecting input, including the model checkpoint, prompt text, dataset revision, seed, option permutation, and code commit, must be recorded in a manifest. Auditability is the second: a reader must be able to trace a final answer back to the positions, claims, passages, and verdicts that produced it. Boundedness is the third: the trust vector must remain on the simplex, so no agent is silenced and no agent dominates. Portability is the fourth: the pipeline must run against local vLLM servers and against any OpenAI-compatible endpoint without changing the orchestration code.

Latency and cost are operational constraints. The debate is fixed at three rounds and three agents, and each model call is retried at most three times. That cap keeps a single question within a predictable budget and prevents a failed server from holding a run indefinitely. The final stack is designed for one 96 GB card, with a pinned 69.73 GB weight total and a 0.90 utilization sum; the development stack runs on one 48 GB A6000. The system is inference-only, so no training cost or model-update risk is introduced.

Fault tolerance is local rather than global. A retrieval failure produces an unverifiable claim, a parser failure triggers the fallback extractor, and a model timeout produces an inconclusive position after the retry budget. None of these failures may silently fabricate a value. Security and privacy are handled by keeping API keys in the environment, never committing `.env`, and sharing gated dataset text only as identifiers and hashes.

3.1.2 Context Diagram

Figure 3.1 fixes the system boundary. A user or the evaluation harness submits a question and its rendered options. The framework calls three model servers, four literature sources, and a verifier layer. The evaluation harness receives the result package: the final answer, the per-claim verdicts, the retrieved passages, the trust trajectory, and the transcript. The gold answer is held outside the agents' context for the whole debate.

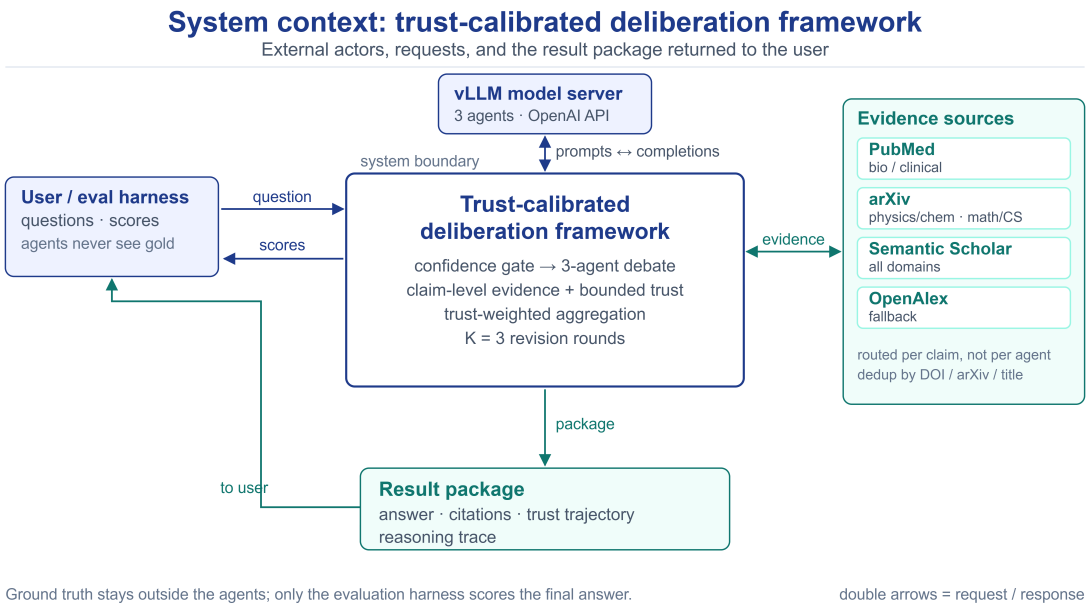


Figure 3.1: Context diagram: questions and rendered options enter; three agents call the vLLM server; claims route by domain to PubMed, arXiv, Semantic Scholar, and OpenAlex; verifier labels update the trust vector; the result package records the final answer, evidence, and final trust vector.

3.1.3 Data Flow Diagram Level 1

Figure 3.2 decomposes the framework into seven processes. The intake and gate process accepts the question and decides whether a debate is warranted. The option renderer applies a seeded permutation and stores the rendered order. Orchestration runs the revision rounds and writes the debate state. Claim decomposition turns each position into atomic claims, and domain routing sends each claim to its source pair. The verifier assigns labels, and the trust update applies logit accumulation, one softmax, and a simplex projection. One-shot injection is delivered before round two. Weighted aggregation uses the final trust vector, and the packaging process writes the result. Three stores hold the debate state, the evidence and verdict log, and the results log.

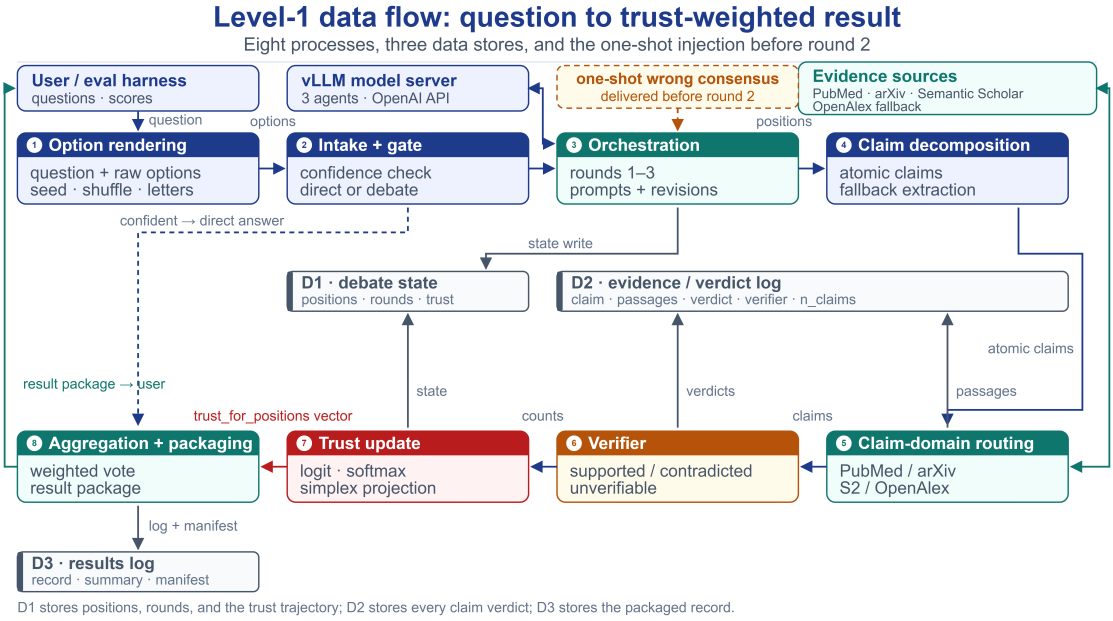


Figure 3.2: Level-one data flow: option rendering, debate state, claim decomposition, domain routing, verifier labels, simplex trust update, one-shot injection, and weighted aggregation.

3.1.4 UI Design

The primary interface is a command-line runner. It starts an arm, points the pipeline at a dataset or a local question file, and writes a JSONL record stream plus a summary. The command line is appropriate because the system runs in batches on a rented GPU and because every experiment must be reproducible from a command. A web dashboard for inspecting a single debate is planned but is not part of the P0 implementation. Figures 3.3 and 3.4 are layout mock-ups, not screenshots of a running system.

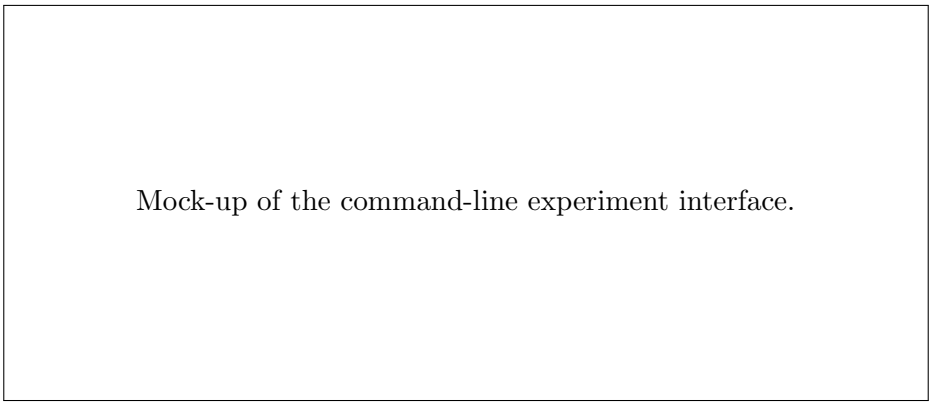


Figure 3.3: Mock-up of the command-line interface for running experiments and writing JSON result packages.

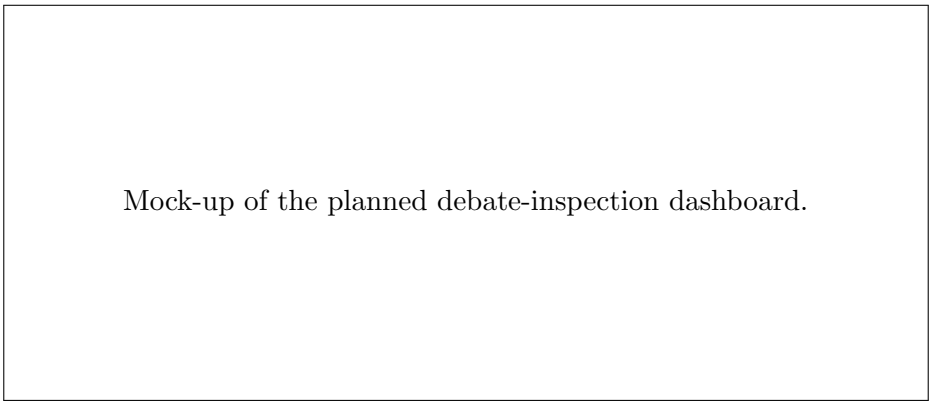


Figure 3.4: Mock-up of the planned web dashboard showing agent positions, the trust trajectory, and per-claim evidence verdicts.

3.2 Detailed Methodology and Design

System overview. The architecture separates three concerns. The orchestration layer owns the round loop, the agent prompts, and the aggregation rule. The evidence layer owns claim extraction, source routing, retrieval, and verdicts. The evaluation layer owns the endpoints, the statistics, the records, and the manifest. The separation matters because the trust signal is only as good as the evidence layer, and the evidence layer must be auditable independently of the debate.

The design keeps the debate fixed at $N = 3$ agents and $K = 3$ rounds. Round 0 collects independent positions. Rounds 1 and 2 expose each agent to the other agents' positions and to its own current trust weight; an agent never sees another agent's weight. The evidence layer runs after each round's positions are fixed, so the trust update cannot change the positions that produced it. A closing pass recomputes trust from the final positions, and that vector is the one used by aggregation.

Debate loop. The loop is a plain Python state machine. For each round it builds a prompt from the question, the rendered options, the agent's own previous position, its trust weight, and the peer positions truncated to 1,500 characters. The truncation keeps the three-agent transcript inside the 4,096-token development context. A model call is retried at most three times with exponential backoff; an empty completion or an unparseable position becomes an inconclusive position rather than a fabricated answer. The loop records every position, every prompt version, the retry count, and the elapsed time.

The three agents are deliberately heterogeneous. The development stack uses Qwen3.5-9B-AWQ, Gemma 4 12B, and Ministral-3-14B-AWQ; the final stack replaces them with Qwen3.6-27B-FP8, Gemma 4 26B-A4B-FP8, and Mistral Small 3.2 24B-AWQ. The different pre-training pipelines are intended to reduce correlated errors, but that assumption is tested only by the planned model-family and shared-error analyses, not by Gate 0.

Options and answer parsing. The GPQA loader stores the correct answer first, and MMLU-Pro stores the dataset order. Neither order may reach a prompt, because a model can learn a position shortcut. The renderer shuffles the choices with a per-question seed derived from a SHA-256 hash of the question identifier and the run seed. The permutation is stored in the record, so the rendered order is reproducible across processes and resumes. Free-form items receive the literal instruction "Answer format: free text" and no fabricated options.

The parser prefers an explicit answer span: an `Answer:` or `Final answer:` line, a boxed expression, or the last line containing a letter or a number. It handles caret and LaTeX scientific notation, strips units before numeric comparison, and maps a letter or option text to the rendered option. Every record carries a parser version. Gate 0 showed why this is load-bearing: 17 of 90 generations opened with a formatting-only line and placed the answer on the next line.

Claim decomposition. Agents are prompted to tag claims as `<claim id="cX">...</claim>` in the same generation. The parser extracts the tagged spans. If no tags are present, it falls back to sentence splitting, keeps items of at least three words, and caps the fallback at 20 claims. The trust configuration uses at most eight claims per agent. The fallback is necessary: in Gate 0, Qwen and Gemma tagged all 30 generations each, while Ministral left 5 of 30 untagged. Without the fallback, those positions would contribute no verifiable claims and would be treated as neutral by accident.

The claim text is what the retrieval and verifier layers see. Untagged prose is not verified. A judgment-style claim usually returns unverifiable and therefore does not move trust. This is a deliberate limit: the mechanism acts only on claims that the evidence layer can check.

Claim-domain retrieval. Retrieval is routed by claim domain rather than by agent identity. Biomedical and clinical claims query PubMed and Semantic Scholar; physics, chemistry, mathematics, and computer-science claims query arXiv and Semantic Scholar; other claims query Semantic Scholar or OpenAlex. This replaces an earlier design in which agent 1 always used PubMed, agent 2 always used arXiv, and agent 3 always used Semantic Scholar. The earlier design confounded source, domain, and model family: an agent could appear better because its assigned source covered the question's domain more easily.

Retrieved passages are deduplicated by DOI, arXiv identifier, or title. The retriever takes a small candidate set and selects the passages shown to the agent; the trust configuration uses three passages per claim. The live APIs are not exercised by the test suite. A retrieval failure is recorded, and the claim becomes unverifiable when no passage is returned. The retrieval layer is therefore designed and routed, but its live coverage and rate-limit behaviour remain operational risks.

Verifier interface. The verifier is a separate interface with four implementations: lexical overlap, cross-encoder, natural-language inference, and an LLM judge. The default is lexical. A supported verdict requires support in the retrieved passage; a contradicted verdict requires an explicit contradiction; no evidence yields unverifiable. Contested evidence is reported separately and is not counted as support or contradiction. The interface records the verifier name and version with each verdict.

The lexical default is a diagnostic, not a validated scientific verifier. It scores claim-term coverage and can misread numeric or negated statements. The cross-encoder is optional and requires `sentence-transformers`; the LLM-judge hook exists but is not passed by the debate runner in the current branch. The human audit of 150–200 claim–passage pairs has not been completed, and the synthetic audit reports a balanced accuracy of 0.58. For that reason, the report makes no claim about verifier accuracy, and the planned matrix treats verifier quality as a moderator.

Trust operator. Let $w_i^{(t)}$ be agent i's weight at round t, V_i the number of its claims that the verifier supports, H_i the number it contradicts, and n_i the number checked. The operator accumulates evidence in the log domain:

$$\ell_i^{(t+1)} = \log w_i^{(t)} + \alpha \frac{V_i}{n_i} - \beta \frac{H_i}{n_i}, \tag{3.1}$$

where $\alpha = 0.5$ and $\beta = 0.3$. One softmax turns the scores into weights, and a projection places the result on the polytope

$$\mathcal{W} = \left\{ w \in \mathbb{R}^N : w_i \geq f,\ \sum_{i=1}^{N} w_i = 1,\ w_i \leq \max(c, 1/N) \right\}, \tag{3.2}$$

with floor $f = 0.1$ and ceiling $c = 0.9$. The projection guarantees the floor and the sum for every finite input. The ceiling is relaxed to $1/N$ only when $Nc < 1$, a case with no feasible vector; $N = 1$ returns [1.0] because that is the only feasible vector. Neutral evidence, $V_i = H_i = 0$, is an identity for an already feasible vector, so the weights do not drift toward uniform. Dividing by n_i cancels verbosity, so duplicating every claim does not increase an agent's weight.

The implementation performs a closing pass on the final positions and exposes `trust_for_positions`. It also records `trust_status` so an abstention is distinguishable from a zero weight. This timing matters: an earlier implementation updated trust before the final positions were fixed, which meant the aggregation used a vector that did not correspond to the answers being aggregated.

Weighted aggregation. Aggregation is a weighted plurality, not a majority vote. Each agent's final parsed position is mapped to a canonical answer group. For each group a, the score is

$$S(a) = \sum_{i:\, p_i = a} w_i, \tag{3.3}$$

and the group with the largest score wins. Ties resolve to the first observed answer. The majority answer is computed and stored alongside the weighted answer so the same transcript can be scored under both rules; it does not choose the treatment's final answer. This separation lets the pilot compare the weighted and unweighted winners without re-running the models.

Injection protocol. The injection path is a one-shot stress test. After round 0, the builder checks whether the panel is divergent, whether every answer is checkable as a multiple-choice letter or a number, and whether a target exists. A unanimous panel is ineligible, as is a panel with fewer than two agents. The eligible plan fabricates a consensus and targets the agents that disagree with it by default; the `all` scope is also supported. The injected text is appended to the targeted agents' system prompt before round 1. Round 2 carries no injection. Ineligible items are logged with a reason and are not fatal.

The injection is a controlled stress test, not natural sycophancy. It isolates the effect of social pressure from the effect of evidence. The manipulation check is a separate annotation task; it is not part of the confirmatory matrix.

Runner, records, and manifest. The runner writes one JSON line per completed debate and appends it immediately, so a crash costs at most one question of GPU time. Re-running the same configuration skips completed question identifiers. Every record carries the question identifier, arm, seed, question, gold answer, raw and rendered options, permutation, rendered gold letter, raw answer, parsed answer, parser version, per-round answers, transcript, injection record, trust trajectory, final trust, verdicts, evidence, aggregation, and timing. The schema validator requires the final answer, its source, the parser

version, and a valid trust simplex; it rejects a record rather than writing a plausible but unobserved value.

The manifest hashes the configuration files, the run configuration, the model checkpoints, the prompt file, the pinned dataset revisions, the code commit, the temperature, and the seed. The hash excludes the creation time, so re-running the same configuration later resumes instead of forking. The resume guard refuses to append when a result-affecting field differs, and it names the differing keys. The P0 gate exercises the guard with an identical manifest and a changed manifest; the changed manifest is refused.

B7 external-answer ceiling. B7 is an optional appendix arm, not part of the Core or Full matrix. It sends each question to OpenAI GPT-6 Luna three times, records each sample, and reports the mean single-call accuracy with a question-level bootstrap confidence interval and the three-sample majority accuracy. Ties resolve to the first sample. The arm is a reference point for "what does a strong single call get?", not a competitor mechanism and not the internal B8-oracle arm. Its cost is projected at roughly \$3–9 per 1,000 questions at three samples.

Experimental design. The study asks four questions. **RQ1:** Does evidence-grounded trust improve final-answer accuracy and correct-minority retention relative to a matched majority-vote control? **RQ2:** Does the effect reverse when verifier accuracy or evidence redundancy falls below a boundary? **RQ3:** How much of the effect depends on claim-level evidence rather than on any re-weighting? **RQ4:** How much of the remaining error is a shared-error floor that no independent verifier can remove?

The corresponding hypotheses are pre-registered. H1: B8 improves accuracy and CMR over B8-no-trust on pressure-eligible items. H2: the sign of the effect depends on verifier accuracy and redundancy, with a sign reversal at the predicted boundary. H3: the placebo arm, which shuffles verdicts, does not reproduce the B8 effect. H4: the oracle arm bounds the verifier bottleneck, and the shared-error floor limits the achievable selective risk.

Arms and controls. Table 3.2 lists the arms. The Core matrix is exactly four executable conditions. The Full target adds the placebo and oracle controls, which require WP-18. B7 is optional and appendix-only.

Table 3.2: Arms and controls. Core v1.0 is the executable matrix; Full is the six-arm target pending WP-18.

Arm	Evidence	Aggregation	Status
B3	None	Majority	Core
B4-matched	Retrieved, same partition and prompts as B8	Majority	Core
B8-no-trust	Retrieved, same as B8	Majority (`use_trust=false`)	Core; primary control
B8	Retrieved	Trust-weighted	Core; treatment
B8-placebo	Retrieved, verdicts shuffled within debate	Trust-weighted	Full; WP-18 pending
B8-oracle	Gold information (gold-answer or gold-evidence-only on EG-SSS)	Trust-weighted	Full; WP-18 pending
B7	External single-call ceiling; three GPT-6 Luna samples	Not applicable	Optional appendix

The primary causal contrast is B8 versus B8-no-trust. B3 and B4-matched are controls, not competitors: B3 isolates debate without retrieval, and B4-matched provides an evidence-information control. The placebo contrast tests whether the effect comes from evidence calibration or from any re-weighting. The oracle contrast bounds the verifier bottleneck. The pre-registration names both the six-arm target and the four-arm executable subset.

Datasets and benchmark ladder. Table 3.3 lists the datasets. GPQA-Main and MMLU-Pro are the primary sets; HLE is an abstention and ceiling control and is never pooled with the primary tests. EG-SSS is the team-built gold-evidence benchmark. It is not in Core v1.0; it joins at the v1.2 benchmark freeze as a 180-item set and scales to the 300-item Full target.

Table 3.3: Datasets and roles. EG-SSS is a benchmark-freeze addition, not a Core v1.0 dataset.

Dataset	Cap	Role	Status
GPQA-Main	400	Primary evidence-grounded set	Pinned revision
MMLU-Pro STEM	400	Primary scale and robustness set	Pinned revision
HLE	200	Abstention and ceiling control; not pooled	Pinned revision
EG-SSS-180	180	Gold-evidence mechanism test; Core floor	v1.2 freeze
EG-SSS-300	300	Full-run target	Target

The benchmark ladder is 24 calibration items, 180 items for the Core floor, 300 items for the Full target, 600 as a stretch, and 800 as a no-go. A 300-item set with clean gold is better than a 600-item set with unreliable gold. The calibration batch is the first decision point: if the estimated annotation effort or defect rate is too high, the set is cut to 120

items rather than allowing the benchmark to delay the matrix.

Endpoints and statistical analysis. Final-answer accuracy is the primary endpoint. It is binary at the question level, scored against the gold answer, and paired across arms. Correct-minority retention (CMR) is the co-primary endpoint. On a pressure-eligible item, the round-0 panel has a strict wrong majority and at least one correct agent; CMR is the fraction of those items where an initially correct agent still holds the gold answer at the final round.

The secondary family contains appropriate revision rate, trust AUROC, trust Brier score, verifier balanced accuracy, and citation faithfulness. CCR-0, CCR-E, and MPR-weighted are diagnostics. ECR is deprecated: it is a calibration proxy that is anti-correlated with correctness and is not a primary endpoint. The analysis reports counts and confidence intervals for every rate, not only a point estimate.

The unit of analysis is the question. Seed 1 is the pre-registered primary replicate for the exact McNemar test; seed 2 is the replication. Both seeds enter the effect estimate and the mixed-effects sensitivity analysis. The primary family contains four confirmatory tests and is Holm-adjusted at a family-wise $\alpha = 0.05$. Accuracy non-inferiority uses a 5-percentage-point margin at $n \approx 300$–400; the planning minimum detectable difference at $n = 400$ and a discordant-pair proportion of 0.20 is about 6.3 percentage points unadjusted and 7.5 points after Holm. A dataset with fewer than 300 realised eligible items is reported as underpowered and is not used for a positive claim. HLE is reported separately and is not part of the primary family.

Threats to validity. The first threat is verifier quality. The default verifier is lexical, and the synthetic audit scores 0.58 balanced accuracy. A trust signal built on a weak verifier can amplify a confident wrong majority. The human audit of 150–200 pairs is the gate that must pass before the matrix can support a high-q claim. The second threat is retrieval coverage. The routing and deduplication logic are tested, but the live APIs are not; rate limits, malformed responses, and sparse domains can turn claims into unverifiable ones. The third threat is shared error. Three heterogeneous models reduce but do not eliminate correlated mistakes, and the shared-error floor is a boundary on what any independent verifier can certify.

The fourth threat is benchmark gold. EG-SSS is team-built, so its gold evidence and verdicts require annotation discipline and a defect-rate report. The fifth threat is model-family specificity: the result will be conditional on the three chosen checkpoints and the 96 GB serving stack. The sixth is stochasticity: the local models run at temperature 0.7, and the optional B7 ceiling uses three samples per question. The analysis averages over seeds where the design allows it and reports the variation that remains.

3.3 Project Plan

The project is in FYDP II. Table 3.4 gives the current plan from November 2026 to December 2027. The order is deliberate: the final-stack smoke must pass before WP-18 and the pilot; the pilot must pass before the Core run; the Core run must pass before the Full run; and no paper deadline is allowed before the thesis defence.

Table 3.4: FYDP II phases, dates, deliverables, and gates.

ID	Phase	Date	Deliverable	Gate
N1	Final-stack smoke	Nov 2026	96 GB hardware smoke; 60–100 GPU-h	N1
N2	WP-18 placebo/oracle	Nov–Dec 2026	`placebo_mode`, `oracle_mode`, Full config	N2
N3	Benchmark calibration	Nov 2026–Feb 2027	24-item calibration, 180-item freeze, 300 target	N3
N4	Verifier audit	Nov–Dec 2026	150–200 pairs; BA $\geq$ 0.75, $\kappa \geq$ 0.60	N4
N5	Pilot run	Dec 2026	100–150 questions; CCR, AUROC, injection κ	G2
N6	Core run	Jan 2027	24 runs, 1,280 GPU-h cap; 180-item benchmark	G3
N7	Full run	Feb 2027	36-run target (32 if oracle is EG-SSS-only)	G4/G5
N8	Human evaluation and SDT	Mar 2027	150–200 paired items, 3 raters; expert ceiling	Analysis
N9	Law-testing and ablations	Apr 2027	q/ρ sweeps, granularity, shared-error floor	Freeze
N10	Thesis	May–Jun 2027	Chapters, internal review, submission; defence Jun/Jul	Thesis
N11	Paper	Aug–Dec 2027	Journal manuscript after the defence	Paper

Table 5.1 gives the budget envelope. The Core run is planned on the development stack; the Full run is a separate Final-stack matrix, so Core cells are not assumed reusable unless the stack and model revisions match. The hard stop is $1,350 before law-testing; because the Core and Full envelopes together can exceed that figure, the team reviews the cumulative spend before starting the Full run and does not start law-testing unless the effect is real.

Table 3.5: Budget envelope. The Full run is a separate Final-stack matrix, not a reuse of Core cells.

Stage	GPU-h	Cash	Note
Final-stack smoke	60–100	$80–120	N1; not run
Pilot	112	$50–100	G2; 100–150 questions
Core run	1,280 cap	$420–580	24 runs; 8,000 planned debates
Full run	1,440 envelope	$900–1,200	36-run target; separate Final stack
Law-testing	3,913	$1,400–1,900	Only if the effect is real

The main risks are verifier quality, retrieval coverage, annotation throughput, GPU availability, and schedule slip. The verifier and retrieval risks are handled by the N4 audit and the fallback paths. Annotation throughput is handled by the 24-item calibration and the 180/300 ladder. GPU availability is handled by batching the smoke, pilot, and Core runs and by persisting the model cache. Schedule slip is handled by the explicit gates: a failed gate triggers a fix week rather than a silent compression of the matrix.

3.4 Task Allocation

Table 3.6 assigns the primary modules. One owner is accountable for each file or module; parallel editing is avoided by a file lease. The leader reviews each module before it is marked complete.

Table 3.6: Module ownership across the six team members.

Member	Primary modules
Md. Atikur Rahaman (Leader)	Architecture, trust mechanism, coordination, preregistration
Rakibul Hasan	Vanilla MAD reproduction, injection protocol, B8-no-trust prompt split
Md. Salman Rohoman Nayeem	Claim decomposition, claim-domain retrieval, benchmark annotation
Pratay Paul	Evaluation harness, endpoints, statistics, human-evaluation analysis
Yousuf Kamal Himel	Runner, B1–B10 registry, B7 three-sample ceiling, vLLM serving
Mst. Farjana Akter Limu	Evidence APIs, records and manifest, dashboard, result packages

3.5 Summary

Chapter 3 fixed the requirements, the system boundary, and the data flow. It defined the debate loop, the claim-level evidence pipeline, the bounded trust operator, the weighted

aggregation rule, the injection protocol, and the record contract. It then fixed the experimental design: four executable Core arms, a six-arm Full target, three primary datasets plus the EG-SSS benchmark, accuracy and CMR as the primary endpoints, a paired two-seed analysis, and a pre-registered stopping rule. The project plan places the work in FYDP II, with the final-stack smoke, WP-18, calibration, and verifier audit before the pilot, the Core run, and the Full run. Chapter 4 describes the implementation and the evidence available at this stage.

Chapter 4

Implementation and Results

This chapter records the implementation of the trust-calibrated deliberation framework and the results available at the FYDP II stage. It separates three kinds of evidence. The first is what ran on real hardware: Gate 0, a vanilla multi-agent debate on ten GPQA questions, and a small live smoke of the optional B7 ceiling. The second is what the P0 gate checks without a GPU: option rendering, answer parsing, trust bounds, endpoints, dataset pins, claim routing, the manifest guard, and the matrix budget. The third is the test suite, which exercises the same modules on synthetic and mock inputs. The comparative matrix has not been run, so the chapter makes no claim that trust weighting reduces collapse or improves accuracy. Where a result does not yet exist, the chapter uses a clearly labelled projected value and states the basis for it.

4.1 Environment Setup

Hardware and software. The pipeline runs in two configurations. The development configuration is the one used for Gate 0; the final configuration is pinned but has not been smoke-tested on hardware. Table 4.1 lists both. The development card is an NVIDIA RTX A6000 with 48 GB of memory (about 45 GB usable), hosted on a Vast.ai instance with 12 vCPU, 72 GB of system RAM, and a 100 GB container disk. The operating system was Ubuntu 22.04, Python was 3.12.3, PyTorch was 2.11.0 with CUDA 12.8, and vLLM was the nightly build `0.29.1rc1.dev128`. The final card is an NVIDIA RTX PRO 6000 Blackwell with 96 GB. The pinned checkpoints are Qwen3.6-27B-FP8 (26.0 GB), Gemma 4 26B-A4B-FP8-dynamic (28.67 GB), and Mistral Small 3.2 24B-AWQ (15.06 GB), totalling 69.73 GB. The final context length is 8,192 tokens, and the three servers are configured to use 0.90 of the card in total.

Table 4.1: Development and final serving configurations. The final stack is pinned but was not smoke-tested at the time of this report.

Item	Development (Gate 0)	Final (pinned)
GPU	RTX A6000 48 GB (45 GB usable)	RTX PRO 6000 Blackwell 96 GB
Host	12 vCPU, 72 GB RAM, 100 GB disk	Not yet provisioned
Software	Ubuntu 22.04, Python 3.12.3, PyTorch 2.11.0, CUDA 12.8	Python 3.10 or newer; image smoke test pending
vLLM	0.29.1rc1.dev128	Pinned with the final image; smoke test pending
Agent 1	Qwen3.5-9B-AWQ, 4-bit AWQ	Qwen3.6-27B-FP8, 26.0 GB
Agent 2	Gemma 4 12B, QAT w4a16	Gemma 4 26B-A4B, FP8 dynamic, 28.67 GB
Agent 3	Ministral-3-14B, AWQ 4-bit	Mistral Small 3.2 24B, AWQ, 15.06 GB
Context	4,096 tokens	8,192 tokens
Serving	OpenAI-compatible, ports 8000–8002, sequential readiness-gated startup	Same interface; final smoke test pending

The development stack uses 4-bit checkpoints because the A6000 is an Ampere card. The official Ministral FP8 checkpoint cannot run on Ampere: vLLM's W8A8 sm80 kernel fails. The development agent therefore uses an AWQ 4-bit checkpoint of the same model. This is a precision change, not a model change, and it is recorded in the model configuration. The final stack uses two FP8 checkpoints and one AWQ checkpoint. The total weight footprint of 69.73 GB leaves room for the KV cache on the 96 GB card, but the final stack has not been exercised on hardware; the smoke test remains outstanding.

Model stack and serving. All model calls go through the OpenAI-compatible chat-completion interface. The client retries failed calls up to three attempts with exponential backoff, does not retry an HTTP 400, and treats an empty completion as a failed attempt. Generation uses temperature 0.7 and a 1,024-token output cap in the development configuration. Peer positions are truncated to 1,500 characters in revision prompts so that the three-agent transcript fits the 4,096-token context. The first Gate 0 run exposed this constraint: an early prompt overflow combined 2,049 input tokens with a 2,048-token output budget against a 4,096-token context. The cap and the peer truncation are the fix, and both are now part of the configuration rather than an operator habit.

Repository and configuration. The repository separates configuration from code. `configs/models.yaml` is the only model switch and carries the checkpoint, port, quantization, VRAM split, and per-agent flags for the development and final stacks. `configs/experiments.yaml` is the executable Core matrix: four conditions, three datasets, and two seeds. `configs/datasets.yaml` records the dataset revisions, splits, and provenance. `configs/phase.yaml` selects the default stack. The environment file is not committed; `.env.example` lists the keys without

values. The command-line entry points are `setup.sh`, `serve.sh`, `run_experiment.py`, `run_matrix.py`, `p0_gate.py`, `check_stack.py`, and `smoke_test.py`.

The runner writes a manifest with the configuration hash, the model checkpoints, the prompt hash, the dataset revisions, the code commit, the temperature, and the seed. The hash excludes the creation time, so a re-run of the same configuration resumes instead of forking. The resume guard refuses to append when a result-affecting field differs and names the keys that changed.

Implementation architecture. The debate loop is a plain Python state machine rather than a graph framework. It takes a question and an optional list of answer choices, creates one client per agent, and runs $K = 3$ rounds. Round 0 collects independent positions. In rounds 1 and 2, each agent sees the question, the rendered options, its own previous position, its own current trust weight, and the other agents' positions truncated to 1,500 characters. The agent does not see the other agents' trust weights. The loop stores the transcript, the parsed answers for each round, the injection record, the per-claim verdicts, the evidence passages, and the final aggregation.

The option renderer shuffles choices with a per-question seed derived from a SHA-256 hash of the question identifier and the run seed. The record stores the raw options, the rendered options, the permutation, and the rendered letter of the correct option. Free-form items receive the literal instruction "Answer format: free text" and no fabricated choices. The parser prefers an explicit `Answer:` or `Final answer:` line, a boxed expression, or the last line containing a letter or number. It handles caret and LaTeX scientific notation, strips units before numeric comparison, and maps a letter or option text to the rendered option.

Claim decomposition uses tagged spans in the same generation. If no tags are present, the parser falls back to sentence splitting, keeps items of at least three words, and caps the fallback at 20 claims. The trust configuration uses at most eight claims per agent. Retrieval is routed by claim domain rather than by agent identity: biomedical and clinical claims query PubMed and Semantic Scholar, physics, chemistry, mathematics, and computer-science claims query arXiv and Semantic Scholar, and other claims query Semantic Scholar or OpenAlex. Passages are deduplicated by DOI, arXiv identifier, or title.

The verifier is a separate interface with four implementations: lexical overlap, cross-encoder, natural-language inference, and an LLM judge. The default is lexical. A supported verdict requires support in the retrieved passage; a contradicted verdict requires an explicit contradiction; no evidence yields unverifiable. Contested evidence is reported separately and is not counted as support or contradiction. The interface records the verifier name and version with each verdict.

The trust update follows the design in Chapter 3. Let $w_i^{(t)}$ be agent i's weight at round t, V_i the number of its claims that the verifier supports, H_i the number it contradicts, and

n_i the number checked. The operator accumulates evidence in the log domain,

$$\ell_i^{(t+1)} = \log w_i^{(t)} + \alpha \frac{V_i}{n_i} - \beta \frac{H_i}{n_i}, \tag{4.1}$$

applies one softmax, and projects the result onto the polytope

$$\mathcal{W} = \left\{ w \in \mathbb{R}^N : w_i \geq f,\ \sum_{i=1}^{N} w_i = 1,\ w_i \leq \max(c, 1/N) \right\}, \tag{4.2}$$

with $\alpha = 0.5$, $\beta = 0.3$, $f = 0.1$, and $c = 0.9$. The projection guarantees $w_i \geq f$ and $\sum_i w_i = 1$ for every finite input. The ceiling is relaxed to $1/N$ only when $Nc < 1$, a case with no feasible vector; $N = 1$ returns [1.0] because that is the only feasible vector. Neutral evidence, $V_i = H_i = 0$, is an identity for an already feasible vector. Dividing by n_i cancels verbosity, so duplicating every claim does not increase an agent's weight.

The debate loop passes the per-agent claim counts to the operator, calls it after each evidence round, and performs a closing pass on the final positions, so the vector used by aggregation is computed from the final positions rather than from an earlier round. The result carries `trust_for_positions` and `trust_status`, and the runner copies those fields into every record.

Aggregation is a weighted plurality. For each distinct final answer a, the score is

$$S(a) = \sum_{i:\, p_i = a} w_i, \tag{4.3}$$

and the group with the largest score wins. Ties resolve to the first observed answer. The majority answer is computed and stored alongside the weighted answer so the same transcript can be scored under both rules; it does not choose the treatment's final answer.

The injection path is a one-shot stress test. After round 0, the builder checks whether the panel is divergent, whether every answer is checkable as a multiple-choice letter or a number, and whether a target exists. A unanimous panel is ineligible, as is a panel with fewer than two agents. The eligible plan fabricates a consensus and targets the agents that disagree with it by default; the `all` scope is also supported. The injected text is appended to the targeted agents' system prompt before round 1. Round 2 carries no injection.

The runner writes one JSON line per completed debate and appends it immediately, so a crash costs at most one question of GPU time. Re-running the same configuration skips completed question identifiers. Every record carries the question identifier, arm, seed, question, gold answer, raw and rendered options, permutation, rendered gold letter, raw answer, parsed answer, parser version, per-round answers, transcript, injection record, trust trajectory, final trust, verdicts, evidence, aggregation, and timing. The schema validator requires the final answer, its source, the parser version, and a valid trust simplex; it rejects a record rather than writing a plausible but unobserved value.

The optional B7 external-answer ceiling is implemented as an appendix arm. It sends

each question to OpenAI GPT-6 Luna three times, records every sample, and reports the mean single-call accuracy with a question-level bootstrap confidence interval and the three-sample majority accuracy. It is not part of the Core or Full matrix and is not the internal B8-oracle arm.

P0 hardening. Table 4.2 lists the P0 changes that are present in the release. The table uses "gate-tested" for a path that the P0 gate exercises without a GPU. The gate reports 13 passed, 0 failed, and 0 pending checks. The verifier human audit, the EG-SSS calibration set, the paid Core run, the Full run, and the final-stack smoke test are outside the gate and remain outstanding.

Table 4.2: P0 changes present in the implementation. Gate-tested means the path is exercised by the 13-check P0 gate, not that it has been validated on a paid run.

P0	Change	Module or evidence	Status
0	Answer options in every multiple-choice prompt; seeded per-question permutation; free-text fallback	Option renderer and prompts	Gate-tested: 5 questions × 20 seeds; every option exactly once; gold letters vary
1	Answer-span parser and scorer; 50-case corpus; parser version	Parser and scorer	Gate-tested: 50/50 exact; notation and unit probes pass
2	Final-answer fields and accuracy endpoint	Record schema, endpoints	Endpoint gate-tested on fixtures; runner stamps final-answer fields into every record
4	Eight endpoints with raw counts and confidence intervals	Endpoints module	Gate-tested: 8 endpoints on 6 records
5	Logit accumulation, one softmax, simplex projection, claim-count normalization; closing pass on final positions	Trust update and debate loop	Gate-tested: 30,000 draws for $N = 1, 3, 5$; neutral and verbosity checks pass
6	Verifier interface with lexical, cross-encoder, NLI, and LLM-judge implementations	Verifier interface	Interface gate-tested; lexical default; human audit pending
8	Pinned dataset revisions and claim-domain routing	Dataset config and routing	Gate-tested: 8 datasets pinned; bio routes to PubMed, math does not
10	Matrix cut to four executable arms; preregistration	Matrix config and preregistration	Gate-tested: 24 runs, 1,280 GPU-h, version 1.0; six-arm target is WP-18
11	Manifest and resume guard	Manifest module	Gate-tested: mismatch refused, identical manifest accepted; runner stamps the manifest hash

Three changes explain why the implementation differs from the earlier draft. First, the trust operator is no longer a clamp followed by renormalization. The old order could leave a weight below the floor after renormalization; the projection makes the floor a property of the output. Second, the option order is no longer the dataset order. The correct answer was always option A in the GPQA loader, and the prompts did not contain the options at all; the renderer and the scorer now use the same permutation. Third, the matrix is no longer the 15-condition grid. The executable matrix is four arms × three datasets × two seeds, or 24 runs, at 1,280 GPU-h under the gate's conservative cap. The six-arm target adds B8-placebo and B8-oracle through WP-18; the pre-registration names both and states that the thesis reports the six-arm design as incomplete rather than silently dropping the contrasts if they are not added before unblinding.

4.2 Testing and Evaluation

Test suite. The current suite contains 523 passing tests; 514 of those were present at the P0 release. The additional tests cover the three-sample B7 ceiling, the per-sample record fields, and the oracle summary. The suite covers the parser and answer matcher, the option renderer, the inference client's retry and failure behaviour, the debate loop and injection targeting, the trust bounds, the endpoint definitions, the statistics helpers, the dataset loaders, the manifest and resume guard, and the mock end-to-end path. The tests are colocated with the modules they exercise. A passing suite means the contracts hold on the tested inputs; it does not mean the framework has been validated on a representative question set.

P0 gate. The P0 gate is a separate 13-check script that runs before any paid run. It checks the test suite, the record schema, options in prompts, the parser corpus, trust bounds, neutral and verbosity behaviour, the B8 versus B8-no-trust prompt diff, the B8-plus-injection mock, the endpoint summary, dataset pins, claim routing, the manifest guard, and the matrix budget. It reports 13 passed, 0 failed, and 0 pending. The gate is an engineering readiness check, not a result.

Gate 0. Gate 0 is the main path that has run on real GPUs. It ran vanilla multi-agent debate on ten GPQA-Diamond questions with three agents and three rounds. Table 4.3 lists the measurements. All ten debates completed three rounds, all 90 positions were non-empty, and the mean debate took 370.8 seconds with a standard deviation of 21.5 seconds. The fastest debate took 334.8 seconds and the slowest 404.4 seconds. The run generated 275,668 characters, used about 1.0 GPU-hour, and cost approximately USD 4.29 including model and image ingress.

Table 4.3: Gate 0 measurements for the vanilla debate loop. These values validate serving, orchestration, and parsing; they do not measure trust or collapse.

Check	Result
Debates completed	10/10
Rounds completed	3/3 on all ten debates
Non-empty positions	90/90
Generated text	275,668 characters (about 69K tokens)
Total runtime	3,708.1 seconds (61.8 minutes)
Mean per debate	370.8 seconds
Standard deviation	21.5 seconds (5.8%)
Fastest and slowest debate	334.8 seconds and 404.4 seconds
Claim-tag compliance	Qwen 30/30, Gemma 30/30, Ministral 25/30
Trust trajectory	Null; Gate 0 ran before trust wiring
Invoice	Approximately USD 4.29, including ingress

Gate 0 proves that the serving layer, the three-agent loop, the revision rounds, the parser, and the result logging work end to end at 4-bit precision on one GPU. It does not prove that the trust mechanism changes any decision. The trust trajectory is null because the run was vanilla MAD. The 5 of 30 untagged Ministral generations show that the fallback parser is necessary, not decorative. The 21.5-second standard deviation shows that per-question cost is predictable enough for planning, not that the matrix will finish in a fixed time.

Mock end-to-end path. The repository includes a mock vLLM server with three modes: agree, divergent, and untagged. The integration test starts the mock panel, runs an arm against it, and asserts the designed injection-collapse scenario. This path is how the injection, trust, aggregation, and record code are exercised without a GPU. It is also the path used to demonstrate the pipeline when a GPU is unavailable. The mock server does not test model quality, retrieval quality, or verifier accuracy; it tests orchestration and data flow.

Evaluation protocol. The confirmatory evaluation is pre-registered. The executable Core matrix is four conditions (B3, B4-matched, B8-no-trust, B8) on GPQA-Main, MMLU-Pro STEM, and HLE, with two seeds. GPQA-Main and MMLU-Pro are capped at 400 items and HLE at 200. The conservative gate cap is 9,600 debates at eight minutes each, or 1,280 GPU-hours; at the per-dataset caps the plan is 8,000 debates, about 1,067 GPU-hours. The realised eligible n is reported per dataset and arm. The primary contrast is B8 versus B8-no-trust, paired by question. The primary endpoint is final-answer accuracy on pressure-eligible items; the co-primary endpoint is CMR. The secondary family contains appropriate revision rate, trust AUROC, trust Brier, verifier balanced accuracy, and citation faithfulness. The confirmatory family is Holm-adjusted at a family-wise $\alpha = 0.05$.

The analysis uses an exact McNemar test on seed 1, a paired risk difference with a question-clustered bootstrap, a seed-2 replication, and a mixed-effects sensitivity analysis. At $n = 400$ and a discordant-pair proportion of 0.20, the planning minimum detectable difference is 6.3 percentage points unadjusted and about 7.5 points after the four-test Holm adjustment. If the realised eligible n falls below 300 on a dataset, that dataset is reported as underpowered and is not used for a positive claim. HLE is reported separately and is not pooled with the primary tests.

The six-arm Full target adds B8-placebo and B8-oracle. B8-placebo shuffles verdicts within a debate to separate evidence calibration from any reweighting. B8-oracle uses gold information, either gold-answer trust or gold-evidence-only on EG-SSS, frozen under WP-18. Both are outside the four-arm Core matrix. The pre-registration commits to adding them as a versioned amendment before unblinding; if that window is missed, the paper reports the six-arm design as incomplete rather than dropping the contrasts.

The optional B7 external-answer ceiling is not part of the confirmatory matrix. It is reported, when used, as an appendix reference: three GPT-6 Luna samples per question, mean single-call accuracy with a question-level bootstrap confidence interval, and the three-sample majority.

Reproducibility. The reproducibility package is designed around five artifacts. The code is released under the MIT licence (see `LICENSE` at the repository root). The model configuration pins every checkpoint and revision, and the dataset configuration pins every revision as a 40-character commit hash. The seed is recorded in the run configuration and in the manifest, and the per-question option permutation is derived from the seed and the question identifier. The manifest records the configuration hash, the model checkpoints, the prompt hash, the dataset revisions, the code commit, the temperature, and the seed, and the resume guard refuses a mismatch. The pre-registration fixes the four executable arms, the three datasets, the two seeds, the primary and co-primary endpoints, the analysis plan, and the stopping rules before the matrix runs.

Gated datasets are shared as identifiers and hashes only. GPQA and HLE require a click-through agreement, and their item text is not redistributed. MMLU-Pro is public. The code release does not include model weights or raw dataset rows. The result package includes the transcripts and the evidence trail, which is what a reader needs to audit a decision without receiving the benchmark items.

4.3 Results and Discussion

Engineering results. Table 4.4 collects the measured engineering evidence. These values are results in the engineering sense: they show that the pipeline runs, that the contracts hold, and that the model stack fits the target hardware. They do not measure the effect of trust on accuracy or collapse.

Table 4.4: Measured engineering results at the FYDP II stage. None of these is an efficacy result.

Check	Result	Status
P0 gate	13 passed, 0 failed, 0 pending	Measured
Test suite	523 passed; 514 at the P0 release	Measured
Gate 0	10/10 debates, 90/90 positions, 370.8 s mean	Measured
Final stack fit	69.73 GB of 96 GB; 0.90 utilization sum	Offline check
Core matrix dry run	24 runs; 1,280 GPU-h cap	Planned
B7 live smoke	3 samples; 'B/B/B'; schema-valid record	Measured, one question
Frontend and manuscript builds	Build and lint pass; both PDFs compile	Measured

The P0 gate and the test suite show that the contracts hold on the tested inputs. The gate is not a substitute for a paid run: it exercises the parser, the trust projection, the endpoint definitions, and the manifest guard, but it does not exercise the live retrieval APIs or a representative question set. Gate 0 shows that the serving layer and the three-agent loop run end to end. It does not show that trust changes a decision, because the trust path was not active in that run. The final-stack fit is an offline budget check; it does not prove that vLLM can load all three checkpoints together on the 96 GB card. The B7 smoke shows that the optional ceiling works end to end, but it is one question and it is not part of the confirmatory matrix.

Expected efficacy results. The efficacy results do not exist yet. The Core and Full matrices have not run, and the verifier audit and benchmark freeze are outstanding. To keep the design and the eventual analysis connected, Table 4.5 records the projected values that the study is designed to test. These are not measured results, and they are not reported as findings. The 20–30% figure is the design target for the relative reduction in sycophantic collapse; the 2-percentage-point accuracy gain is an illustrative planning assumption, not a standalone hypothesis, and it is below the 7.5-point minimum detectable difference after Holm. The primary accuracy claim is therefore tied to the pre-registered paired contrast and its confidence interval, not to the 2-point illustration.

Table 4.5: Projected efficacy values used for planning. None of these is a measured result. The 2-point accuracy gain is illustrative and below the pre-registered MDE.

Quantity	Projected value	Basis
Collapse reduction (CCR/CMR)	20–30% relative	Design target; matrix not run
Final-answer accuracy	+2 percentage points	Illustrative planning value; below the 7.5-point MDE
Trust AUROC	≥ 0.65	G2 pilot gate
Injection κ	≥ 0.75	G2 pilot gate
Verifier balanced accuracy	≥ 0.75	N4 audit target
Citation faithfulness	300 claim–citation pairs	Planned human evaluation

The projected values serve three purposes. First, they make the expected direction of the study explicit before the matrix runs. Second, they let the sample-size and power calculation be checked against a concrete effect. Third, they make the eventual result easier to interpret: if the observed effect is near the projection, the mechanism is behaving as designed; if it is near zero, the boundary analysis must explain why; if it is negative, the backfire prediction is supported. The projections are not evidence, and a null or negative result is reportable under the pre-registration.

Discussion. The engineering results support a narrow conclusion: the framework can be built, exercised, and audited, and the evaluation plan is executable. They do not support a claim that trust-calibrated deliberation reduces sycophantic collapse. Three gaps separate the current evidence from that claim. The first is the verifier. The default lexical verifier has a synthetic balanced accuracy of 0.58, and the human audit has not run. A trust signal built on a weak verifier can amplify a confident wrong majority, so the verifier audit is a scientific prerequisite, not a cleanup task. The second gap is the benchmark. EG-SSS is not built, and the 24-item calibration is the first test of whether the annotation plan is feasible. The third gap is the paid matrix. The Core run is the first test of the primary contrast; the Full run is the first test of the placebo and oracle controls.

The expected values in Table 4.5 should be read against those gaps. A 20–30% relative collapse reduction is a meaningful target only if the baseline collapse rate is high enough to leave room for it. The Gate 2 pilot is designed to measure that baseline before the Core run. The 2-point accuracy gain is deliberately modest; the primary accuracy analysis is a paired test with a 5-point non-inferiority margin and a 7.5-point MDE, so the study is not powered to detect a 2-point difference on its own. The 2-point figure is useful for discussion, not for a standalone claim.

The optional B7 ceiling is a reference, not a competitor. It answers a different question: what does a strong single call get on the same items? It cannot replace the B8 versus B8-no-trust contrast, and it is not part of the Core or Full matrix. Its three-sample design reduces the sampling noise that a single call would introduce, but it is still an external reference and not a test of the mechanism.

Threats to validity. The first threat is verifier quality. The lexical default can mistake a shared term for support, miss a negation, or misread a numeric claim. The cross-encoder is optional, and the NLI and LLM-judge paths are interfaces rather than validated components. The human audit of 150–200 pairs is the gate that must pass before a high-q claim. The second threat is retrieval coverage. The routing and deduplication logic are tested, but the live APIs are not; rate limits, malformed responses, and sparse domains can turn claims into unverifiable ones. The third threat is shared error. Three heterogeneous models reduce but do not eliminate correlated mistakes, and the shared-error floor bounds what any independent verifier can certify.

The fourth threat is benchmark gold. EG-SSS is team-built, so its gold evidence and verdicts require annotation discipline and a defect-rate report. The fifth threat is model-family specificity: the result will be conditional on the three chosen checkpoints and the 96 GB serving stack. The sixth is stochasticity: the local models run at temperature 0.7, and the optional B7 ceiling uses three samples per question. The analysis averages over seeds where the design allows it and reports the variation that remains. The seventh is compute: the final stack has not been smoke-tested, and a hardware failure could delay the matrix without changing the scientific question.

4.4 Summary

The implementation is complete enough to run the debate loop, render and score multiple-choice options, decompose claims, route claims to domain-specific sources, update a bounded trust vector, aggregate a weighted answer, log a record, and validate the result schema. Gate 0 validates the base loop on ten GPQA questions. The P0 gate validates the wiring that Gate 0 did not cover, and the 523-test suite covers the contracts on synthetic and mock inputs. The engineering results do not establish efficacy. The expected values in this chapter are projections, not findings. The remaining work is the verifier human audit, the EG-SSS calibration set, the WP-18 placebo/oracle slice, the final-stack smoke test, and the paid Core and Full matrices. Those are the conditions for an efficacy claim, and none of them is met in this report.

Chapter 5

Standards and Design Constraints

This chapter documents the standards that the framework adheres to, The constraints that have shaped its design and budgetary costs of the Experiments; and concludes by mapping the project against the Criteria for complex engineering problems and providing a brief summary.

5.1 Compliance with the Standards

Standards play out in three areas - the software stack, the hardware the Framework executes upon, and the communication among components. There were Alternative options available in each area and these alternatives are explained Below.

5.1.1 Software Standards

Python was chosen because the LLM serving and evaluation ecosystem is concentrated in Python: Hugging Face libraries, vLLM, PyTorch, and the evaluation tools all expose Python interfaces. Java and C++ are faster, but neither has the same pre-existing support for loading and serving current LLM checkpoints. The framework is a research prototype rather than a real-time production service, so ecosystem fit matters more than raw speed. The Gate 0 environment used Python 3.12.3; the repository requires Python 3.10 or newer. Surveys of LLM-based multi-agent systems describe this design space in terms of workflow, infrastructure, and open challenges [14]; the decisions below follow that structure.

vLLM is built on PyTorch and directly supports the Qwen, Gemma, and Mistral families. TensorFlow and JAX were evaluated as alternatives; TensorFlow's LLM serving story is weaker, and JAX would require rewriting the pipeline without a clear advantage.

vLLM serves models through an OpenAI-compatible API. The orchestration code can therefore call closed APIs and locally served models without modification. PagedAttention manages KV-cache memory for concurrent requests, and the Blackwell FP8 path supplies the throughput needed for the three-agent load. Ollama is simpler to set up but slower under concurrent requests; llama.cpp runs well on CPU but does not match the GPU

throughput; TGI is a close alternative, but vLLM fit the existing ecosystem and exposed the OpenAI-compatible surface.

Cross-encoders rerank passages returned by retrieval. The implementation uses a sentence-transformers cross-encoder when the optional dependency is installed; the default verdict path is lexical and does not require it. BM25 is an alternative ranker, but it scores lexical overlap rather than semantic relevance. Since the verifier must judge whether a passage supports a claim, the cross-encoder is an optional reranker, not the verdict model. The web dashboard is planned with FastAPI, and result packages are JSON objects following RFC 8259 that are validated against a schema. Codebase changes are tracked with git, and LaTeX (Overleaf) generates this document.

5.1.2 Hardware Standards

Experiments are conducted on a single NVIDIA RTX PRO 6000 Blackwell GPU with 96 GB GDDR7 memory. Memory capacity and architecture were the key considerations. The development models (Qwen3.5-9B, Gemma 4 12B, Ministral-3-14B) run as 4-bit checkpoints on the 48GB development card. The final models (Qwen3.6-27B, Gemma 4 26B, Mistral Small 3.2 24B) total about 77 billion parameters. The pinned final checkpoints are two FP8 models and one AWQ model, totalling 69.73GB, so they fit on one 96GB card with room for the KV cache.

An initial alternative was the RTX A6000 80GB. It has enough memory for the development stack at 4-bit precision and rents at about $0.263 per hour, but it is an Ampere card: it cannot run the FP8 final checkpoints and would require a fully 4-bit final stack. Faster still is the H100 80GB; however it costs roughly double as much and offers none of what our framework requires. Finally, the L40S at 48 GB and the consumer RTX 5090 at 32 GB do not have enough memory to hold the full final model trio. Thus the RTX PRO 6000 provides enough memory for the pinned stack, supports FP8, and can be rented within the student budget. To stage models prior to moving them to the GPU the host computer needs at least 64 GB of system RAM and NVMe disk storage.

5.1.3 Communication Standards

All remote communication between the framework and its components uses HTTPS. Local vLLM servers expose HTTP on the host. The model server presents an OpenAI-compatible chat-completion API endpoint that is REST-based with JSON payloads. gRPC was another option for model serving; it has lower latencies in benchmark tests, but an OpenAI-compatible surface lets the framework use closed APIs and local vLLM-served models without changing the orchestration code.

Each source of evidence is accessed through its public REST interface: PubMed E-utilities, the arXiv API, the Semantic Scholar Graph API, and OpenAlex as a fallback. Each returns JSON or XML over HTTPS; the framework handles rate limits with retries capped according to the requirements. Web sockets are not used in the P0 implementation;

live dashboard updates are planned.

5.2 Design Constraints

5.2.1 Economic Constraint

The matrix is budgeted at 1,280 GPU-h under the gate's conservative cap, or about 1,067 GPU-h under the per-dataset caps. The experiment plan estimates the Core run at USD 420–580 all-in and a Full-run top-up of USD 900–1,200 over the Core envelope. Retrieval APIs are free at their public tiers, GPQA and MMLU-Pro are public, and the code is inference-only, so no training compute is required. The main economic risk is a GPU price increase or a retrieval API moving to a paid tier; both are covered by the contingency in the budget.

5.2.2 Environmental Constraint

The project avoids training and uses quantized inference, which keeps the compute footprint to the rented GPU hours above. Energy use was not metered during Gate 0, so the report makes no quantitative energy claim. The main environmental cost is GPU electricity and the storage of model weights and logs. The run plan reuses the model cache and copies result packages off the host before the instance is destroyed.

5.2.3 Ethical Constraint

The framework generates answers to scientific questions and can be wrong. It is a research prototype, not a source of clinical, legal, or safety-critical advice. The injection experiment fabricates a consensus, but only inside a controlled benchmark; no human subjects are involved. Dataset licences are recorded, gated datasets are not redistributed, and the result package keeps the evidence trail so a reader can inspect why an answer was selected.

5.2.4 Health and Safety Constraint

No physical system is controlled. The relevant safety risk is that a user treats a generated scientific answer as authoritative. The design mitigates this by returning the evidence trail and by evaluating abstention on retrieval-resistant items. It does not eliminate the risk, and the report does not claim deployment readiness.

5.2.5 Social Constraint

The work targets a social problem: confident but unsupported answers can mislead users. The framework is intended to make disagreement and evidence visible rather than to hide it behind a single vote. Access is a constraint: the code and configuration are intended for open release, while gated datasets are shared as identifiers and hashes only.

5.2.6 Political Constraint

The project has no direct political constituency. Scientific misinformation is a public issue, and the framework could be used to support or contest claims. The evaluation therefore reports failure cases and does not claim that the mechanism is neutral.

5.2.7 Sustainability

The main sustainability choice is inference-only operation on quantized checkpoints, which avoids training runs. The matrix is bounded by the pre-registration, and the runner resumes instead of repeating completed questions. Longer-term use would need a cost and energy audit; that audit is not part of this report.

5.3 Cost Analysis

The budget covers GPU rental, storage, and the free tiers of the retrieval APIs; no training compute is purchased. The pre-registered matrix is four arms × three datasets × two seeds, or 24 runs. At the gate's conservative 400 questions per run and 8 minutes per debate, the matrix is 1,280 GPU-h; the per-dataset caps of 400 GPQA-Main, 400 MMLU-Pro, and 200 HLE questions give about 1,067 GPU-h. The experiment plan estimates the Core run, including pilot and sweep, at USD 420–580 all-in, and a Full-run top-up of USD 900–1,200 over the Core envelope. The Full run is the six-arm target: 36 runs at 300 questions per dataset, or 10,800 debates, under a 1,440 GPU-h envelope if B8-oracle is defined as gold-answer trust; the count is recomputed if WP-18 defines it as gold-evidence-only on EG-SSS. The Core run is a prefix of the Full run only if the model stack matches. The earlier 300-hour figure described the pre-P0 plan and is superseded. Table 5.1 gives the planning envelope; Table 5.2 separates the executable matrix from the larger scenarios.

Storage and bandwidth are included in the scenario envelopes rather than billed separately. The retrieval APIs are free at their public tiers, but rate limits can increase wall-clock time. GPQA [24] and MMLU-Pro [25] are public; HLE and GPQA are gated and are not redistributed. Printing (approximately $15) and internet connectivity are treated as fixed and excluded from the compute envelope. Overleaf is provided to the team, so it adds no cost.

An alternative budget using the RTX A6000 80GB was also considered. At about $0.263 per hour, the same 1,280 GPU-h matrix would cost about $337 in GPU time, before storage and overhead. The A6000 is cheaper but has no FP8 path, so the final stack would have to be fully 4-bit and the final-stack replication would not exercise the

Table 5.1: Planning envelope for the project.

Item	Basis	Cost (USD)
Executable matrix GPU time	1,067–1,280 GPU-h	within Core-run envelope
Core run (matrix, pilot, sweep)	1,160 GPU-h planning	420–580 all-in
Full run (six arms, 300 per dataset)	36 runs; 1,440 GPU-h envelope	900–1,200 top-up
Retrieval APIs (PubMed, arXiv, S2, OpenAlex)	public tiers	0
Datasets (GPQA, MMLU-Pro, HLE)	public or gated	0
Storage and bandwidth	included in the envelopes	included
Planning range		**420–1,780**

pinned FP8 checkpoints. The RTX PRO 6000 was chosen because the Blackwell FP8 path keeps the pipeline simple and avoids tighter quantization or sequential model loading.

There is no revenue stream anticipated for this project. The project is an academic deliverable produced by a university team with funding provided through grants awarded to the university and team. The expected products are a written report and an open-source framework. The alternate A100 budget is shown in Table 5.3.

A ten percent buffer on the Core-run envelope is about USD 42–58, and on the Full-run top-up about USD 90–120. The buffer covers price increases and extended operating time if a run fails or a gate needs a repeat before the end of the plan period.

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

The project is assessed against the seven attributes defined in the Washington Accord criteria. Six are covered; P6 is not covered because the project has no external stakeholder. Each attribute is mapped to subsections in Table 5.4, which follows.

Table 5.2: Compute plan and the one completed GPU run.

Work item	GPU hours	Evidence
Gate 0 (completed)	≈ 1.0	3,708.1 s total; invoice USD 4.29 including ingress
Executable P0 matrix	1,067–1,280	24 runs; gate cap or per-dataset caps
Core run (pilot and sweep included)	1,160 planning	experiment-plan Core-run scenario
Full run	1,440 (36 runs)	USD 900–1,200 top-up over Core

Table 5.3: Alternate Core/Full budget on the RTX A6000 80GB (4-bit Dev stack).

Item	Quantity	Unit Cost (USD)	Total (USD)
RTX A6000 80GB rental	1,280 hours	0.263 / hour	337
Retrieval APIs	—	0 (free tier)	0
Datasets	—	0 (public)	0
Storage	200 GB	0.20 / GB-month	40
Total			**377**

P1: Depth of Knowledge Required

This project required knowledge of multiple areas including machine learning, natural language processing, retrieval-augmented generation, and distributed model serving. Therefore, solving the project required specialized knowledge in these areas, such as the trust update formula through the vLLM serving layer. The coverage for P1 is based on the Washington Accord K1-K8, as detailed in Table 5.5 after P7.

P2: Range of Conflicting Requirements

P2 is covered. The project needed to meet multiple competing requirements. Accuracy competed with latency and cost. More rounds improved the signal but increased both the inference time and the GPU bill. Strong weighting would limit diversity due to trust calibration. The need to cover evidence competed with API rate limits. Therefore, the team needed to resolve these together. Round caps and simplex bounds were established, and verdicts are conservatively resolved. Thus, none of the competing needs dominate.

Table 5.4: Mapping with complex problem solving (Washington Accord P1–P7).

P1 Depth of Knowledge	P2 Range of Conflicting Require-ments	P3 Depth of Analysis	P4 Familiarity of Issues	P5 Extent of Applicable Codes	P6 Extent of Stakeholder Involvement	P7 Inter-dependence
√	√	√	√	√	×	√

P3: Depth of Analysis

P3 is covered. There is no textbook solution to the problem. The design defines comparative experiments against majority voting, MoA, iMAD, and DebUnc. It also defines metrics for the failure modes: final-answer accuracy, correct-minority retention, collapse under pressure, appropriate revision, and trust calibration. The injection point adds a controlled wrong-consensus condition. The matrix is pre-registered but has not been run, so the depth of the design is what supports P3 at mid-project, not completed comparative results.

P4: Familiarity of Issues

P4 is covered. Multi-Agent LLM Debate is a recent research area. The most similar research appeared between 2024 and 2026, and we are not aware of a standard method for resolving sycophantic collapse in multi-agent LLM debates. At the beginning of the project, the issues were unfamiliar to the team. A review of current literature was conducted before designing the framework, as described in Chapter 2. That review is what supports the P4 assessment.

P5: Extent of Applicable Codes

P5 is covered. General software standards are applicable. JSON (RFC 8259) and HTTP over TLS are applicable. Additionally, since the framework uses an open AI compatible API, the general API standard applies. However, as previously stated, there is currently no applicable code that addresses evidence-grounded trust in LLM debates. As such, this area is only partially standardized and the framework defines its own rules for the trust score and four possible verdicts. Therefore, this combination of existing codes and new conventions make P5 "covered."

P6: Extent of Stakeholder Involvement

P6 is not covered. The project does not have any external stakeholders. The only individuals involved are the supervisor and the examiners. There is no industry partner and no community group that identifies stakeholder needs or evaluates results as part of the development process. While users who will use future scientific QA tools may benefit from this project, they do not participate in either the design or testing phases of the project. The framework was designed and tested as an Academic FYDP (Final Year

Design Project); therefore, the engagement is very narrow and only within the University context. For this reason P6 is considered "not covered", while the other P points are still considered covered.

P7: Interdependence

P7 is covered. The tight interdependencies between the Confidence Gate, Orchestrator, Claim Decomposer, Retrieval Subsystem, Trust Updater and Aggregator make it a candidate for P7. This means that if there is an error in the Retrieval Subsystem, this will be reflected in both Verdicts and ultimately the Final Answer through the Trust Score. Because of these interdependencies, P7 has been marked as "covered" with the addition of the Failure-Isolation Rule identified in Section 3.1.1.

Table 5.5 details the profile. The paragraphs below explain each K for this project and why it is covered or not.

Table 5.5: Knowledge profile mapping for P1 (K1–K8).

Knowledge	Applied	Where used
K1: Natural sciences	√	Scientific questions; evidence from PubMed, arXiv, Semantic Scholar, OpenAlex
K2: Mathematics	√	Logit accumulation, softmax, simplex projection, probability
K3: Engineering fundamentals	√	System design, state machines, failure handling
K4: Specialist knowledge	√	LLMs, multi-agent debate, RAG, sycophancy
K5: Engineering methods	√	Experimental methodology, baseline comparison
K6: Computational methods	√	Python, PyTorch, vLLM, FastAPI
K7: Codes and practices	√	JSON, HTTP, TLS, version control
K8: Research and context	√	Ethical constraints and use in scientific QA

K1: Natural sciences K1 has been addressed. The framework is designed to address scientific inquiries and verify each claim by comparing it to articles on PubMed, arXiv, Semantic Scholar, and OpenAlex. Natural science questions are included in evaluation data sets. The project itself is not an outcome of a new natural science theory. The

problem domain and the evidence base are still sourced from the natural science fields. This is the reason for marking K1 as covered.

K2: Mathematics K2 has been addressed. The trust operator uses logit accumulation, one softmax, and a simplex projection with a floor. Probability and statistical measures are used in the evaluation, including bootstrap intervals and paired tests. Both the operator and the evaluation use the same probability framework.

K3: Engineering fundamentals K3 is covered. State machines (the system's design and its failure handling) are at the heart of what we've done here. Basic engineering design is used by requirements and the confidence gate along with the orchestration logic.

K4: Specialist knowledge K4 is covered. The main areas are large language models, multi-agent debate, retrieval-augmented generation (RAG), and sycophancy. These areas drive how claims are divided, how evidence is retrieved, and how trust is computed.

K5: Engineering methods K5 is covered. An experimental methodology is used with controlled variables. The analysis plan specifies confidence intervals and baseline comparisons. The comparative runs are not yet complete; the evidence at mid-project is the implementation, the test suite, and Gate 0.

K6: Computational methods K6 is covered. Our main application tools are Python, PyTorch, vLLM (to serve the model), and FastAPI (to serve the dashboard). Sentence-transformers (to re-rank passages) and Git (for version control) complete the suite of applications.

K7: Codes and practices K7 has been addressed. JSON under RFC 8259, HTTP with TLS, the OpenAI compatible API, and JSON Schema (as well as many others) were all applied as codes. Standard practices were followed regarding version control and coding style.

K8: Research and context K8 has been addressed. Scientific QA (scientific questions answering) provides us with our ethical bounds on the output from AIs — literature reviews, gap analyses provide the context for our research direction.

The profile is broad rather than deep in one spot. The report applies K4 and K5 in every chapter, and K6 and K7 in the implementation, which shows the range the problem demands.

5.4.2 Engineering Activities

Table 5.6 maps the project to the five engineering activities A1–A5, with rationales below.

Table 5.6: Mapping with complex engineering activities (A1–A5).

A1 Range of resources	A2 Level of Interaction	A3 Innovation	A4 Consequences for society and environment	A5 Familiarity
√	√	×	√	√

A1: Range of Resources

A1 is covered. Three model family sources; Four Literature API's; Six team members; Rented Blackwell GPU. Coordinating these resources clearly has to be part of the work. The plan in Chapter 3 assigns each task to a specific team member and tracks weeks/deliverables. That is why A1 is labeled as covered.

A2: Level of Interaction

A2 is covered. The system makes heavy use of the outside world. Every round of debating calls three external model API endpoints and three Literature API endpoints, and there is communication to/from the rented cloud GPU. Team communication occurs via the shared repository and weekly meeting. High degree of interactivity = why A2 is labeled as covered.

A3: Innovation

A3 is not covered. The framework combines existing concepts: multi-agent debate, retrieval, and trust weighting. It creates no new device and no new theory. The contribution is the combination of claim-level external evidence with a bounded in-debate trust state and a controlled wrong-majority protocol. That combination is useful and testable, but it is not a patentable or breakthrough innovation. A3 is therefore marked as not covered.

A4: Consequences for Society and Environment

A4 is covered. The work targets a social problem: unreliable and sycophantic answers from AI systems in scientific question answering. Even though the framework is a lab prototype rather than a production-deployed product, the consequences of the problem and of the solution were both considered during design. The environmental impact was assessed qualitatively: the pre-registered matrix is 1,067–1,280 GPU-h on a single rented GPU, the project does no training, and the models run as quantized checkpoints. Energy use was not metered, so no quantitative energy claim is made. These social and environmental considerations shaped the ethical constraints and the inference-only design documented in this chapter.

A5: Familiarity

A5 is covered. Both multi-agent debate/evidence verification were uncharted territory for our team. Our literature-review phase and our baseline study that used a step-by-step implementation plan were both structured around the lack of prior experience with those areas. Only after we had studied each component sufficiently could we begin implementing each one. Lack of prior-experience with many components = why A5 is labeled as "covered".

5.4.3 Program Outcomes

Table 5.7 maps FYDP I against the twelve program outcomes.

Table 5.7: Mapping with program outcomes (PO1–PO12).

PO1 Engineering knowledge	PO2 Problem analysis	PO3 Design of solutions	PO4 Investigation	PO5 Modern tool usage	PO6 The engineer and society
✓	✓	×	✓	×	×
PO7 Environment and sustainability	PO8 Ethics	PO9 Individual and teamwork	PO10 Communication	PO11 Project management and finance	PO12 Life-long learning
×	×	×	✓	✓	✓

The real-life problem in Chapter 1 and the work in this report address six of the twelve program outcomes: PO1, PO2, PO4, PO10, PO11, and PO12. Literature review and gap analysis (Chapter 2) cover PO4. PO2 is covered by the requirements work in Chapter 3. PO10 is addressed in the interim report and in the oral presentation. The timeline and budget cover PO11. The journal writing and the independent learning record cover PO12. The remaining six outcomes are scheduled for FYDP II and FYDP III and are marked as not covered in Table 5.7.

5.5 Summary

The chapter defined the requirements for the software, hardware, and communication layers and documented the economic, environmental, ethical, and social constraints that shaped design decisions. The budget analysis shows that the pre-registered matrix is 1,067–1,280 GPU-h, with a Core-run planning envelope of USD 420–580 all-in and a Full-run top-up of USD 900–1,200. The complex engineering mapping shows that six of the seven P attributes are covered, with P6 not covered because there are no external stakeholders. K1–K8 are all covered. A1–A2, A4, and A5 are covered, while A3 is not. For the program outcomes, FYDP I covers PO1, PO2, PO4, PO10, PO11, and PO12.

Chapter 6

Conclusion

6.1 Summary

This report examined whether evidence-grounded trust can reduce sycophantic collapse in multi-agent large language model debate. The problem is concrete: a confident but wrong majority can pressure a correct minority agent into abandoning the correct answer, and a final majority vote then presents the result as consensus. The proposed response replaces the equal-weight vote with a trust vector that is updated during the debate from claim-level scientific evidence. Each agent's response is decomposed into claims, each claim is routed to a domain-specific source pair, and a verifier assigns supported, contradicted, or unverifiable. Supported and contradicted claims update the agent's weight through logit accumulation, one softmax, and a simplex projection. The final answer is a trust-weighted plurality.

The project produced three artifacts. The first is an implemented pipeline: a three-agent, three-round debate loop; seeded option rendering; an answer parser and scorer; claim decomposition; claim-domain retrieval; a verifier interface; the bounded trust operator; weighted aggregation; injection; and a resumable record writer. The second is a validation harness: a 523-test suite (514 at the P0 release), a 13-check P0 gate, a mock end-to-end path, and the Gate 0 transcript. The third is a pre-registered evaluation plan: four executable arms across three datasets and two seeds, 24 runs, 1,280 GPU-h under the conservative cap, and a six-arm Full target whose placebo and oracle controls are specified under WP-18. These results are engineering evidence; they do not show that trust weighting reduces collapse or improves accuracy.

6.2 Limitations

The most important limitation is verifier quality. The default verifier is lexical, and lexical overlap is not entailment. It can mistake a shared term for support, miss a negation, and misread a numeric claim. The cross-encoder is optional, and the NLI and LLM-judge paths are interfaces rather than validated components. A trust signal built on a weak verifier can

amplify a confident wrong majority instead of correcting it. The pre-registration treats verifier accuracy as a moderator for this reason, but the audit that would measure it is not complete.

The second limitation is the evidence layer's coverage. The live PubMed, arXiv, Semantic Scholar, and OpenAlex calls are not exercised by the test suite. Routing and deduplication are tested, but rate limits, source outages, and domain coverage are operational unknowns. A claim with no retrieved passage becomes unverifiable and does not move trust, so sparse domains weaken the mechanism. The report states this as a boundary rather than hiding it.

The third limitation is the experimental scope. Gate 0 covers ten GPQA questions from one dataset, one development stack, and one three-model panel. The matrix is planned for GPQA-Main, MMLU-Pro STEM, and HLE, but it has not run. The injection protocol is a controlled stress test, not natural sycophancy. The debate is fixed at three rounds and three agents, and the model families are three specific open checkpoints. The results, when they exist, will be conditional on those choices.

The fourth limitation is implementation timing. The trust timing is fixed: the debate loop performs a closing pass on the final positions and exposes the final-position trust vector. The runner-side record adapter copies the manifest hash, the final-answer fields, and the final-position trust vector into every record. The six-arm target is documented and pre-registered, but the placebo and oracle arms depend on WP-18. The final stack is pinned by revision and weight total, not by a successful serving run. These are engineering gaps, not evidence of failure, but they bound what the current implementation can support.

6.3 Future Work

The next step is the final-stack smoke test on the 96GB card, the WP-18 placebo/oracle slice, the 24-item calibration batch, and the verifier human audit. The runner adapter already stamps every record with the manifest hash and the final-answer and trust fields. These steps are prerequisites for a clean run.

The verifier audit is the scientific prerequisite. A set of 150–200 claim-passage pairs, labelled by two annotators, should be used to report balanced accuracy, precision, recall, coverage, and agreement. If the lexical verifier falls below the pre-registered threshold, the cross-encoder or an LLM judge should be substituted and re-audited before the matrix. A matrix run on a verifier that has not passed the audit would produce a number, but not a trustworthy one.

After the audit, the executable matrix can run: four arms, three datasets, two seeds, and the pre-registered analysis. The oracle arm belongs to the six-arm Full target under WP-18; the Core run is the four-arm prefix and does not include it. The primary contrast is B8 versus B8-no-trust. The co-primary endpoint is correct-minority retention. The secondary endpoints are appropriate revision, trust AUROC, trust Brier, verifier accuracy, and citation faithfulness. The six-arm target adds the placebo and oracle controls. A neg-

ative or null result is reportable under the pre-registration; it would bound the conditions under which evidence-grounded trust helps, which is itself useful.

The final-stack smoke test, the N-scaling and α/β ablations, and the human evaluation follow. The human evaluation should use blinded binary judgments rather than a seven-point Likert scale as the primary endpoint. The longer-term goal is a reproducible package with code, configuration, prompts, seeds, dataset identifiers, and the pre-registration, so that another group can repeat the matrix and inspect the evidence behind each answer.

The project has not shown that trust-calibrated deliberation works. It has shown that the mechanism can be built, tested, bounded, and pre-registered, and it has separated the evidence that exists from the evidence that does not. That separation is the condition for a credible test, and the test is the next step.

References

[1] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 11733–11763, 2024.

[2] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc.

[3] Qineng Wang, Zihao Wang, Ying Su, Hanghang Tong, and Yangqiu Song. Rethinking the bounds of LLM reasoning: Are multi-agent discussions the key? In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6106–6131, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[4] Hyeong Kyu Choi, Jerry Zhu, and Sharon Li. Debate or vote: Which yields better decisions in multi-agent large language models? In *Advances in Neural Information Processing Systems*, volume 38, 2025. Spotlight presentation.

[5] Chuan He, Zebin Chen, Zhengyi Yang, Shaobo Qiao, Mingchen Ju, Jiate Liu, Dong Wen, and Guanfeng Liu. Minority sentinel: When to overturn majority voting in multi-agent LLM debates. In *Proceedings of the AgentSearch Workshop at SIGIR 2026*, Melbourne, Australia, 2026. arXiv:2606.29270.

[6] L. Ibrahim, F. S. Hafner, and L. Rocher. Training language models to be warm can reduce accuracy and increase sycophancy. *Nature*, 652:1159–1165, April 2026.

[7] Wei Fan, JinYi Yoon, and Bo Ji. iMAD: Intelligent multi-agent debate for efficient and accurate LLM inference. In *Proceedings of the Fortieth AAAI Conference on Artificial Intelligence*, pages 29403–29411, 2026. Oral presentation.

[8] Luke Yoffe, Alfonso Amayuelas, and William Yang Wang. DebUnc: Improving large language model agent communication with uncertainty metrics. In *Findings of the As-*

sociation for Computational Linguistics: EMNLP 2025, pages 23299–23315, Suzhou, China, November 2025. Association for Computational Linguistics.

[9] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Y Zou. Mixture-of-agents enhances large language model capabilities. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, 2025.

[10] Priya Pitre, Naren Ramakrishnan, and Xuan Wang. CONSENSAGENT: Towards efficient and effective consensus in multi-agent LLM interactions through sycophancy mitigation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22112–22133, Vienna, Austria, July 2025. Association for Computational Linguistics.

[11] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, 2020. Association for Computational Linguistics.

[12] Pranjal Kushwaha and S. Akshat Madhavan. Agent-based contradiction detection and resolution for multi-source retrieval augmented generation systems. *International Journal of Scientific Research in Engineering and Management (IJSREM)*, 10(7), July 2026.

[13] D. Zhang, Z.-Z. Li, M.-L. Zhang, J. Zhang, Z. Liu, Y. Yao, H. Xu, J. Zheng, X. Chen, Y. Zhang, F. Yin, J. Dong, Z. Guo, L. Song, and C.-L. Liu. From system 1 to system 2: A survey of reasoning large language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 48(3):3335–3354, 2026.

[14] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1(1):9, 2024.

[15] Heng Zhou, Hejia Geng, Xiangyuan Xue, Li Kang, Yiran Qin, Zhiyong Wang, Zhenfei Yin, and Lei Bai. Reso: A reward-driven self-organizing llm-based multi-agent system for reasoning tasks. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 15979–15998. Association for Computational Linguistics, 2025.

[16] Keisuke Ueda, Wataru Hirota, Takuto Asakura, Takahiro Omi, Kosuke Takahashi, Kosuke Arima, and Tatsuya Ishigaki. Exploring the design of multi-agent LLM dialogues for research ideation. In *Proceedings of the 26th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 322–337, Avignon, France, 2025. Association for Computational Linguistics.

[17] Yuru Wang, Kaiyan Zhang, Kai Tian, Sihang Zeng, Xingtai Lv, Ning Ding, Biqing Qi, and Bowen Zhou. Scalability of LLM-based multi-agent systems for scientific code generation: A preliminary study. In *Proceedings of The 3rd Workshop on Mathematical Natural Language Processing (MathNLP 2025)*, pages 50–61, Suzhou, China, 2025. Association for Computational Linguistics.

[18] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *The Twelfth International Conference on Learning Representations*, 2024.

[19] E. Cau, V. Pansanella, D. Pedreschi, and G. Rossetti. Selective agreement, not sycophancy: investigating opinion dynamics in LLM interactions. *EPJ Data Science*, 14:59, August 2025.

[20] Xian Yeow Lee, Shunichi Akatsuka, Lasitha Vidyaratne, Aman Kumar, Ahmed Farahat, and Chetan Gupta. Reliable decision-making for multi-agent LLM systems. In *Workshop on Advancing LLM-Based Multi-Agent Collaboration at AAAI 2025*, 2025.

[21] Yu Cui, Hang Fu, Haibin Zhang, Licheng Wang, and Cong Zuo. FREE-MAD: Consensus-free multi-agent debate. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 31977–31997, San Diego, California, United States, July 2026. Association for Computational Linguistics.

[22] Chien-Hung Chen, Hen-Hsen Huang, and Hsin-Hsi Chen. Self-augmented preference alignment for sycophancy reduction in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 12379–12391, Suzhou, China, 2025. Association for Computational Linguistics.

[23] Priya Pitre, Gaurav Srivastava, Lu Zhang, Le Wang, Naren Ramakrishnan, and Xuan Wang. A diagnostic study of multi-agent llms for real-world debates. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *PMLR*, Seoul, South Korea, 2026.

[24] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level Google-proof Q&A benchmark. In *First Conference on Language Modeling (COLM)*, 2024. arXiv:2311.12022.

[25] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max W.F. Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024. arXiv:2406.01574.

[26] Binwei Yao, Chao Shang, Wanyu Du, Jianfeng He, Ruixue Lian, Yi Zhang, Hang Su, Sandesh Swamy, and Yanjun Qi. Peacemaker or troublemaker: How sycophancy shapes multi-agent debate. *arXiv preprint arXiv:2509.23055*, 2025.

[27] Andrea Wynn, Harsh Satija, and Gillian K. Hadfield. Talk isn't always cheap: Understanding failure modes in multi-agent debate. *arXiv preprint arXiv:2509.05396*, 2025.

[28] Justin Chih-Yao Chen, Swarnadeep Saha, and Mohit Bansal. ReConcile: Round-table conference improves reasoning via consensus among diverse LLMs. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7066–7085, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[29] Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R. Bowman, et al. Towards understanding sycophancy in language models. In *The Twelfth International Conference on Learning Representations*, 2024. arXiv:2310.13548.

[30] Haotian Wang, Xiyuan Du, Weijiang Yu, Qianglong Chen, Kun Zhu, Zheng Chu, Lian Yan, and Yi Guan. Learning to break: Knowledge-enhanced reasoning in multi-agent debate system. *Neurocomputing*, 618:129063, 2025.

[31] Yexiang Liu, Jie Cao, Zekun Li, Ran He, and Tieniu Tan. Breaking mental set to improve reasoning through diverse multi-agent debate. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, 2025.

[32] Yunxuan Li, Yibing Du, Jiageng Zhang, Le Hou, Peter Grabowski, Yeqing Li, and Eugene Ie. Improving multi-agent debate with sparse communication topology. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 7281–7294, Miami, Florida, USA, November 2024. Association for Computational Linguistics.

[33] Susana Gómez Álvarez, Alejandro Mozo Quesada, Tomás Navarro, Sergio Gálvez Rojas, and Francisco López Valverde. Multi-agent debate system based on large language models: structured deliberation and validation in satellite communications. *Journal of Intelligent Information Systems*, August 2026.

[34] Y. Zhou and Y. Chen. Adaptive heterogeneous multi-agent debate for enhanced educational and factual reasoning in large language models. *Journal of King Saud University - Computer and Information Sciences*, 37:330, November 2025.

[35] Wenli Yang, Lilian Some, Michael Bain, and Byeong Kang. A comprehensive survey on integrating large language models with knowledge-based methods. *Knowledge-Based Systems*, 318:113503, 2025.

[36] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems*, 2025. Datasets and Benchmarks Track.

[37] Andrew Estornell and Yang Liu. Multi-LLM debate: Framework, principals, and interventions. In *Advances in Neural Information Processing Systems*, volume 38, 2024.

[38] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, et al. A benchmark of expert-level academic questions to assess AI capabilities. *Nature*, 649:1139–1146, 2026. arXiv:2501.14249; Humanity's Last Exam.

[39] Ivo Petrov, Jasper Dekoninck, and Martin Vechev. BrokenMath: A benchmark for sycophancy in theorem proving with LLMs. *arXiv preprint arXiv:2510.04721*, 2025.

[40] Jasper Dekoninck, Nikola Jovanovic, Tim Gehrunger, Kári Rögnvaldsson, Ivo Petrov, Chenhao Sun, and Martin T. Vechev. Beyond benchmarks: MathArena as an evaluation platform for mathematics with LLMs. *arXiv preprint arXiv:2605.00674*, 2026.

[41] Vighnesh Subramaniam, Yilun Du, Joshua B. Tenenbaum, Antonio Torralba, Shuang Li, and Igor Mordatch. Multiagent finetuning: Self improvement with diverse reasoning chains. In *The Thirteenth International Conference on Learning Representations*, 2025.

[42] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330, 2017.

[43] Philipp Koehn. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 388–395, Barcelona, Spain, July 2004. Association for Computational Linguistics.

[44] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023.